

Parametric Maximum Closure on Directed Forests

Computational Report – data release v0.3.0

Valerio Dose

Fabio Furini

Marco Locatelli

1 September 2026

This report accompanies the manuscript “*On parametric Maximum Closure Problems over precedence forests*”. The manuscript states the algorithms, their analysis, and the few experiments needed to support the theory. This report is the complement: the full experimental record, and specifically the material the manuscript does not carry –

- absolute running times of *every* algorithm at *every* size of every campaign, not a selection;
- breakdowns by *arc density* and by *coefficient family*, which is where the interesting structure turns out to be (Section 8.2);
- what the instance generators actually produced – arcs, components, number of closure layers – so that a reader can tell what each parameter buys (Section 5);
- the dispersion of the raw measurements, so that any reported ratio can be compared against the noise it rests on (Section 13);
- the implementation analysis behind the $\mathcal{O}(n)$ space bound (Section 11), and the exact protocol, including what “peak RSS” means and why memory is only quoted from single-algorithm runs.

Every table and plot is generated from the raw benchmark data by `tools/build_report.sh`; no number is hand-typed.

How to read the figures. Every result in this report is presented in the same shape: a table of numbers on the left, a plot of *the same numbers* on the right, and one caption below both. Every caption follows the same three-part order:

1. **what is reported** – the quantity and the statistic, spelled out (for instance “median CPU time in milliseconds, the median over instances of the per-instance median over repetitions”);
2. **on which instances** – the instance class, the sizes, and how many instances each reported value aggregates;
3. **what we observe** – the reading of the figure, and where possible the mechanism behind it.

Unless a caption says otherwise, every time is CPU time in milliseconds, every “median” is taken over instances of the per-instance median over repetitions, and every ratio is a *paired* per-instance ratio (Section 3).

Contents

1	Summary	2
2	The algorithms, in one page	3
3	Definitions and conventions	4
4	Setup	5
5	Instances	5
5.1	How many trees, and how big?	6
5.2	How many closure layers?	7

5.3 How big are the closure layers?	7
6 Validation	7
7 The campaigns	7
8 Random forests	12
8.1 Medium sizes: direct scan versus heap	12
8.2 Large sizes, and the effect of density	13
8.3 What the random campaigns say	18
9 Structured classes	18
9.1 What the structured classes say	18
10 Single orientations	22
10.1 What the oriented classes say	22
11 Implementation note: heap policy	27
12 Comparison with parametric pseudoflow	27
13 Dispersion of the measurements	29
A Per-family detail at every size	29
B Reproducing this report	29

1 Summary

- **Sweep.** One frozen commit, one machine, 2026-08-31/09-01: 16 200 instances, 292 080 timed runs, seven algorithms plus the external BPPF baseline. No run hit the 300 s timeout or the 8 GiB memory ceiling, so nothing is censored.
- **Correctness.** Zero cross-algorithm disagreements (Section 6), on top of an exhaustive enumeration oracle on small instances and sanitizer builds.
- **Random forests, paths, binary trees.** HPaC is the fastest of our algorithms: 2–13× RaC depending on size and density, and faster than the direct scan PaC from $n \approx 400$ on.
- **Stars.** The ordering inverts: RaC is $\approx 250\times$ HPaC at $n = 20\,000$. Any heap-based schedule pays $\Theta(n)$ maintenance per event on a hub; PaC, which maintains no candidate structure, resists an order of magnitude better than HPaC.
- **Single orientations.** HIPaC/HOPaC take 0.52–0.78 of HPaC’s time, stable up to $n = 10^5$.
- **Against pseudoflow.** HPaC is faster than BPPF by a median factor 3.3 on forests with $n \leq 1\,000$, even with the breakpoints handed to BPPF for free; BPPF misses the exact layer sequence on 16 of 2 400 instances, for precision reasons (Section 12).
- **Two facts that organize everything else.** The *topology* decides which algorithm wins – density moves the margin by an order of magnitude and the star class inverts it outright – while the *coefficient family* decides only how much parametric structure exists, moving absolute times but never the ranking. And the data structure inside one algorithm can matter as much as the choice of algorithm: the two heap policies of Section 11 differ by 4× in time and two orders of magnitude in memory, at identical output and identical asymptotic bounds.

2 The algorithms, in one page

All of them solve the same problem and return the same object: given a directed forest with integer profits p_i and strictly positive integer weights w_i , the ordered sequence of closure layers of $u(\lambda) = \max\{P(S) - \lambda W(S) : S \text{ closed}\}$, with the exact rational threshold of each layer. All of ours use exact integer arithmetic: 64-bit coefficients, 128-bit cross-multiplication for every ratio comparison, no floating point in any decision path.

Table 1: The algorithms compared in this report: the class of forests each one applies to, its worst-case time and space bounds, and how it selects the next candidate. Bounds are those proved in the manuscript, except BPPF’s, which is the published bound of the parametric pseudoflow method on a graph with n vertices and m arcs. A forest has $m \leq n - 1$, i.e. $m = \mathcal{O}(n)$, so on *our* inputs BPPF’s bounds specialize to $\mathcal{O}(n^2 \log n)$ time and $\mathcal{O}(n)$ space: its time bound is worse than that of every algorithm above it, which is the theoretical reason a forest-specific method is worth having.

	Applies to	Time	Space	Selection of the best candidate
PaC	any forest	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	linear scan
DPaC	any forest	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	linear scan, dual order
HPaC	any forest	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n)$	two candidate heaps
DHPaC	any forest	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n)$	two heaps, dual order
HIPaC	in-forests	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	one heap over the vertices
HOPaC	out-forests	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	one heap, dual order
RaC	any tree	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	none: top-tree contraction
BPPF	general graphs	$\mathcal{O}(mn \log n)$	$\mathcal{O}(m)$	external, parametric pseudoflow
	<i>on a forest</i>	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n)$	<i>since $m \leq n - 1$</i>

What each one does, and what the experiments below add:

- **PaC** peels the current best final vertices or contracts the best arc, finding the best candidate by a linear scan at every iteration. Slowest on random forests beyond $n \approx 400$, but the most robust of the three on stars, precisely because it maintains nothing.
- **DPaC** is its dual: it builds the sequence from the lowest ratio upwards. It tracks **PaC** everywhere we measured.
- **HPaC** performs the same two moves, but keeps the candidates in two heaps, so only those touched by the last move are refreshed. Fastest of ours on random forests, paths and binary trees; penalized on a high-degree hub, where a single move invalidates a constant fraction of the candidates.
- **DHPaC** is its dual, within 25% of it everywhere.
- **HIPaC** exploits out-degree ≤ 1 : the minimal preceding set of an arc is then a singleton, so one heap value per vertex suffices and no closure sums are propagated. Takes 0.52–0.75 of **HPaC**’s time. **HOPaC** is the symmetric case (in-degree ≤ 1): 0.57–0.78.
- **RaC** contracts the tree in $\mathcal{O}(\log n)$ rake-and-compress rounds, computing cluster functions, then recovers the thresholds top-down. It is the only algorithm whose cost does not depend on how often a single vertex is touched – hence the winner on stars, by up to $250\times$.
- **BPPF** is the external baseline: it solves a strictly more general problem, evaluates minimum cuts at parameter values supplied by the caller, and uses fixed-point arithmetic – hence the precision caveat of Section 12. Its $\mathcal{O}(mn \log n)$ bound is stated for a general precedence graph; a forest has $m \leq n - 1$, so on the instances of this report it reads $\mathcal{O}(n^2 \log n)$: a factor n above the $\mathcal{O}(n \log n)$ of **RaC** and of the single-orientation variants, and a factor $\log n$ above the $\mathcal{O}(n^2)$ of **PaC**. The measured gap of Section 12 should be read against that: it is a factor of a few, far smaller than the gap between the bounds.

Two remarks that matter for reading the results. First, **PaC** and **HPaC** are the *same algorithm* under

two schedules – the difference measured in Section 8.1 is entirely the cost of finding the maximum-ratio candidate. Second, RaC pays a structural overhead per component (cluster bookkeeping) that the others do not, which is why density matters so much in Section 8.2: a sparse forest is many small components.

3 Definitions and conventions

Terms used throughout, fixed here once.

- **Closure layer.** One block $\mathcal{I}_r$ of the canonical partition of the vertex set induced by the parametric solution: the vertices that enter the optimal closed set exactly when λ crosses the breakpoint λ_r . “Number of layers” equals the number of breakpoints of $u(\lambda)$, so it measures how much parametric structure an instance has, and it bounds the number of iterations of every algorithm here.
- **Threshold** of a vertex: the breakpoint at which that vertex enters the optimal closed set; vertices with equal threshold form one layer.
- **Density ϱ .** The probability with which the generator adds each candidate arc. A forest with density ϱ has $\varrho(n-1)$ arcs and $n - \varrho(n-1)$ connected components in expectation, so $\varrho = 1.0$ gives one spanning tree and $\varrho = 0.3$ a forest of many small trees (Fig. 1).
- **Coefficient family.** The rule used to draw the profits and weights of an instance; six are used (Section 5). It controls the *shape* of the ratios p_i/w_i and hence how many layers appear.
- **Paired ratio.** The ratio of two algorithms’ times *on the same instance*. Reported as the median over instances, never as a ratio of two aggregates: pairing removes the instance-to-instance variation that would otherwise swamp the comparison.
- **IQR (interquartile range).** $Q_3 - Q_1$: the width of the band containing the central half of the observations. It is to the median what the standard deviation is to the mean, but insensitive to the tails. Beside a median ratio, a large IQR signals that the population mixes regimes instead of scattering around one value – exactly what happens across densities in Section 8.2.
- **Relative IQR** (Section 13): the IQR of the repetitions of one instance divided by that instance’s median time, in percent. It quantifies measurement noise, not algorithmic variability.
- **Peak RSS** (resident set size): the maximum amount of physical RAM the process had mapped during the run, from `getrusage(RUSAGE_SELF).ru_maxrss`. See Section 4 for the two caveats that make it usable only for single-algorithm runs.
- **Censored run.** A run killed by the time limit (300 s) or the memory ceiling (8 GiB), hence without a measured time. There are none in this study; the term appears only to state that.
- **Campaign.** One preregistered block of the sweep – a set of instances plus the algorithms run on them (B, C, D, E, G).
- **Tree of a forest, and its size.** A connected component of the *underlying undirected* graph, and its number of vertices. Orientations are irrelevant to this notion: a forest of in-trees and a forest of mixed orientations built with the same seed have the same trees. A vertex with no incident arc is a tree of size 1, called *isolated* (Section 5.1).
- **SHA-256 checksum.** A 256-bit fingerprint of a file, computed with the standard SHA-2 hash function: any change to the file’s bytes, even a single one, changes the fingerprint, and no one can construct a different file with the same fingerprint. We record it for every instance file (`instances/manifests/*.json`) and for every archive in the data release, so that a regenerated or downloaded instance can be proved byte-for-byte identical to the one measured here – a reproducibility device, not a security one. It is unrelated to the 64-bit `sequence_hash` used to compare algorithm *outputs* (Section 6), which is a cheap non-cryptographic hash.

4 Setup

Machine and build. Intel Core i7-12700, 32 GB RAM, Ubuntu 22.04.5 (kernel 6.8), GCC 11.4.0 at `-O3`, single-threaded. Each benchmark process pinned to one physical core (`taskset`); the six campaign lanes ran on distinct cores, never two on the same core. CPU governor `powersave` (no passwordless root on this machine), so absolute times carry ordinary frequency-scaling noise – which is one reason every comparison is a paired same-instance ratio.

What is timed. Only the algorithm call (`std::chrono::steady_clock`); parsing, result hashing and correctness comparison sit outside the timed region. Repetitions: 11 for random forests with $n \leq 1000$, 3 elsewhere. Algorithm order shuffled per repetition, so no algorithm is systematically favoured by the CPU frequency state.

Memory, and what “peak RSS” means. RSS is the *resident set size*: the amount of physical RAM the process has mapped at a given instant. Peak RSS is its maximum over the run, read from `getrusage(RUSAGE_SELF).ru_maxrss`. Two properties matter when reading memory figures. It is a *process-lifetime* peak and never decreases, so in an invocation that benchmarks several algorithms the value recorded for the second includes the peak of the first – memory is therefore quoted only from single-algorithm invocations (Section 11). And it counts pages actually resident, so it includes allocator overhead and is precisely the quantity that decides whether a run survives a memory ceiling.

Statistics. Absolute times: median over repetitions, then median over instances. Comparisons: *paired* per instance, reported as the median of the per-instance ratios together with their IQR (Section 3), never a ratio of aggregate means. The median is chosen because it is invariant under inversion of the comparison (the arithmetic mean of ratios is not: it depends on which algorithm sits in the denominator), because the timing noise has an asymmetric right tail by construction (interference can only slow a run down, never speed it up), and because it is the conservative choice on every comparison where we claim an advantage – on the BPPF comparison at $n = 1000$ the three aggregates are 4.5 (median), 5.8 (geometric mean) and 8.7 (arithmetic mean).

5 Instances

An instance of this problem is two independent things: a *precedence structure* (which vertex forces which) and a set of *affine coefficients* (a profit and a weight per vertex). We generate the two separately and cross them, so that any effect observed can be attributed to one or the other. Concretely, one instance is fully determined by four choices: a topology, a coefficient family, a size n , and a random seed. Nothing else is tuned – in particular there is no capacity, no time horizon and no scaling of the coefficients.

Files use the `.pcf` format: the arc list, one integer profit and one strictly positive integer weight per vertex, where arc (u, v) encodes the implication $x_u \leq x_v$. The format has no capacity field and no reader accepts one – a deliberate choice, since the same algorithms exist in the literature for capacitated variants and we did not want the two to be confusable. Every instance is written by a seeded deterministic Python generator, never by hand, and is recorded in a manifest with its SHA-256 checksum, its topology classification and its coefficient bounds; a regenerated or downloaded archive is therefore verifiable byte-for-byte against the one used here.

Below, each of the two components is described with the rule that generates it. Section 5.3 then measures what the combination actually produces – how many closure layers, and how big – which is the quantity that governs how much work every algorithm has to do.

Coefficient families (six).

- **independent-positive**: w_i, p_i independent uniform on $\{1, \dots, 1000\}$.
- **independent-signed**: w_i as above, p_i uniform on $\{-1000, \dots, 1000\}$.
- **correlated**: $p_i = w_i + \delta_i$, δ_i uniform on $\{-50, \dots, 50\}$.
- **anti-correlated**: $p_i = (1001 - w_i) + \delta_i$.
- **near-ties**: one target ratio num/den per instance (num, den uniform on $\{1, \dots, 20\}$), then t_i uniform on $\{1, \dots, 50\}$, $w_i = \text{den } t_i$, $p_i = \text{num } t_i + j_i$ with jitter $j_i \in \{-2, -1, 1, 2\}$.
- **exact-ties**: $g = \max(1, \lfloor n/20 \rfloor)$ groups each with its own exact ratio; vertex i goes to group $i \bmod g$, so equal-threshold vertices are spread over the graph rather than adjacent.

Topologies (six). **mixed-forest**: each $v \geq 2$ linked to a uniform predecessor $u < v$ with probability ϱ , each edge oriented (u, v) or (v, u) with probability $1/2$. **in-forest**: one arc (u, j) with j uniform above u , with probability ϱ (out-degree ≤ 1); **out-forest** symmetric (in-degree ≤ 1). **path-mixed**, **binary-mixed**, **star-mixed**: fixed underlying shape (path; balanced binary tree, parent $\lfloor (v-1)/2 \rfloor$; star, hub 0), orientations randomized.

Sizes and counts. 240 instances per topology and size (four densities $\times$ six families $\times$ ten seeds), 60 for the structured families, which have no density parameter. The random topologies use the ten sizes 100, 200, $\dots$, 1 000 and the ten sizes 10 000, 20 000, $\dots$, 100 000; the structured families use 100, 200, 500, 1 000, 2 000, 5 000, 10 000, 20 000, 50 000 and 100 000.

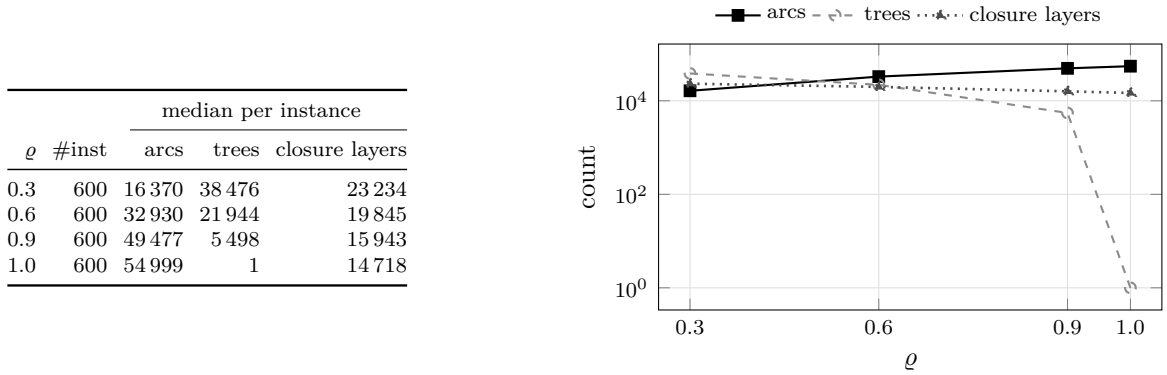


Figure 1: Median number of arcs, of trees and of closure layers of the generated instances, per arc density. Instances: **mixed-forest**, $n \in \{10\,000, \dots, 100\,000\}$ pooled, 600 instances per density. Observations: what the density parameter actually produces. Arcs grow linearly in ϱ and the number of trees falls accordingly, from $\approx 38\,500$ trees at $\varrho = 0.3$ to a single spanning tree at $\varrho = 1.0$. The number of closure layers moves the other way and far less: connecting the forest removes only about a third of them, because precedence constraints merge layers that would otherwise be distinct. This is why density is a genuine experimental factor and not a cosmetic one.

5.1 How many trees, and how big?

Figure 1 counts the trees of the generated forests; this subsection measures how large they are, because “sparse forest” can mean two very different structures – many trees of comparable size, or one giant tree surrounded by dust – and the two would explain RaC’s density sensitivity (Section 8.2) in different ways. The measurement is made directly on the .pcf files by `tools/forest_structure_stats.py`, without running any algorithm: the trees are a property of the input.

Two closed-form facts anchor the numbers. The generator adds each of the $n - 1$ candidate arcs independently with probability ϱ and never creates a cycle, so a forest has $\varrho(n - 1)$ arcs and therefore exactly $n - \varrho(n - 1)$ trees in expectation: the tree count is $(1 - \varrho)n$ to first order, and the *mean* tree size is $1/(1 - \varrho)$. Both are independent of n – so density, not size, is what sets the fragmentation, and the same density means the same fragmentation at $n = 10^3$ and at $n = 10^5$.

5.2 How many closure layers?

The number of closure layers is the number of breakpoints, hence an upper bound on the number of iterations of every peeling algorithm: it is the single best predictor of absolute running time. The two figures below report it per coefficient family and per size.

5.3 How big are the closure layers?

The campaign CSVs record how *many* layers an instance has, not how big they are: the algorithms are timed and their output is discarded. The size distribution is worth one measurement of its own, because it is what a peel step actually consumes. Figure 6 reports it on a sample of 60 instances at $n = 10\,000$ (all six families, four densities, plus stars, paths and in-forests), 261 232 layers in total.

6 Validation

- **Exhaustive enumeration oracle** (CTest): every directed forest with ≤ 4 vertices over a finite coefficient grid; 10 000 random forests up to 11 vertices; mixed-orientation path, binary and star trees up to 257 vertices; 6 000 in-forest and 6 000 out-forest instances plus 2 000 larger ones per orientation; plus the structural invariants (layers partition V , each is a closure increment, thresholds strictly decreasing).
- **Cross-algorithm differential agreement**: every campaign runs several algorithms on the same instance and compares partition, thresholds and canonical order through a 64-bit fingerprint. Over the sweep: *zero* disagreements (Table 2).
- **Sanitizers and CI**: GCC Release, GCC Debug with the address and undefined-behaviour sanitizers, and Clang Debug; all three pass at the frozen commit (`results/TEST_REPORT_2026-08-31.md`).
- **Overflow**: `validate_instance` refuses any instance whose total absolute profit or weight would leave the exact-arithmetic margin (`INT64_MAX/4`), before any algorithm runs.

7 The campaigns

A *campaign* is one block of the sweep: a set of instances, the algorithms run on them, and the number of repetitions. The split is not cosmetic – each campaign answers one question, and keeping them separate is what allows a result to be attributed to a cause:

- **A** is correctness only, on instances small enough to be solved by brute force, so that every algorithm can be checked against an independent oracle rather than against another algorithm.
- **B** and **C** are the same random topology at two scales. B is where all five algorithms still fit, so it is the one place where every pairwise comparison is available; C drops the two quadratic ones and asks how the rest scale.
- **D** fixes the shape (path, balanced binary tree, star) and varies nothing but orientations and coefficients, which isolates the effect of topology – and produces the one case where the theoretical separation between our algorithms becomes visible.

class, n	ϱ	#inst	arcs	#trees	trees/ n	median	mean	largest	% of n	isolated (%)
binary, 10^5	–	10	99 999	1	0.000	100 000	100 000.0	100 000	100.0	0.0
in, 10^5	0.3	10	29 940	70 060	0.701	1	1.4	65	0.1	76.9
in, 10^5	0.6	10	59 989	40 011	0.400	1	2.5	1 196	1.2	62.4
in, 10^5	0.9	10	89 971	10 029	0.100	1	10.0	37 940	37.9	52.6
in, 10^5	1.0	10	99 999	1	0.000	100 000	100 000.0	100 000	100.0	0.0
mixed, 10^3	0.3	10	291	709	0.709	1	1.4	12	1.2	77.9
mixed, 10^3	0.6	10	599	401	0.401	1	2.5	61	6.1	61.7
mixed, 10^3	0.9	10	904	96	0.096	1	10.2	579	57.9	52.4
mixed, 10^3	1.0	10	999	1	0.001	1 000	1 000.0	1 000	100.0	0.0
mixed, 10^4	0.3	10	3 011	6 989	0.699	1	1.4	30	0.3	77.0
mixed, 10^4	0.6	10	6 038	3 962	0.396	1	2.5	338	3.4	62.5
mixed, 10^4	0.9	10	8 991	1 009	0.101	1	9.9	4 448	44.5	53.2
mixed, 10^4	1.0	10	9 999	1	0.000	10 000	10 000.0	10 000	100.0	0.0
mixed, 10^5	0.3	10	29 864	70 136	0.701	1	1.4	59	0.1	76.9
mixed, 10^5	0.6	10	60 000	40 000	0.400	1	2.5	1 561	1.6	62.6
mixed, 10^5	0.9	10	89 942	10 058	0.101	1	10.0	36 056	36.1	52.8
mixed, 10^5	1.0	10	99 999	1	0.000	100 000	100 000.0	100 000	100.0	0.0
out, 10^5	0.3	10	30 000	70 000	0.700	1	1.4	62	0.1	77.0
out, 10^5	0.6	10	59 992	40 008	0.400	1	2.5	1 292	1.3	62.5
out, 10^5	0.9	10	89 968	10 032	0.100	1	10.0	32 324	32.3	52.9
out, 10^5	1.0	10	99 999	1	0.000	100 000	100 000.0	100 000	100.0	0.0
path, 10^5	–	10	99 999	1	0.000	100 000	100 000.0	100 000	100.0	0.0
star, 10^5	–	10	99 999	1	0.000	100 000	100 000.0	100 000	100.0	0.0

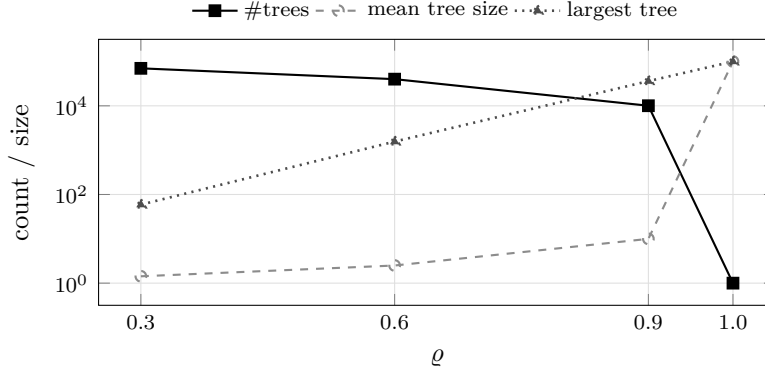


Figure 2: Structure of the generated forests: median number of arcs, median number of trees, trees per vertex, then the median, mean and largest tree size, the largest tree as a percentage of n , and the percentage of trees that are isolated vertices; the plot below the table shows the three quantities that move with the density, at $n = 100\,000$. Instances: **mixed-forest** at $n \in \{10^3, 10^4, 10^5\}$, **in-forest** and **out-forest** at $n = 10^5$, all four densities, plus the three structured classes at $n = 10^5$; 10 instances per row, all with **independent-positive** coefficients, since the topology is generated independently of the coefficients and one family therefore represents all six. Observations: measured values match the closed forms above to three digits – $(1 - \varrho)n$ trees, mean size $1/(1 - \varrho)$ – and are the same for **mixed-forest**, **in-forest** and **out-forest**, confirming that orientation does not touch the structure; **path-**, **binary-** and **star-**mixed are single trees by construction. The interesting column is the largest tree: at $\varrho = 0.3$ and $\varrho = 0.6$ it holds 0.1% and 1.6% of the vertices at $n = 10^5$, but at $\varrho = 0.9$ it holds 36% – while the median tree is *still* a single vertex and half the trees are still isolated vertices.

s	% of trees with size $\geq s$			
	$\varrho = 0.3$	$\varrho = 0.6$	$\varrho = 0.9$	$\varrho = 1.0$
1	100.00	100.00	100.00	100.00
2	23.04	37.48	47.25	100.00
3	8.64	20.47	30.48	100.00
5	2.22	9.30	17.63	100.00
10	0.31	3.06	8.19	100.00
100	0	0.07	0.66	100.00
1000	0	< 0.01	0.07	100.00
10000	0	0	0.01	100.00
largest tree	96	2866	42153	100000
#inst	10	10	10	10

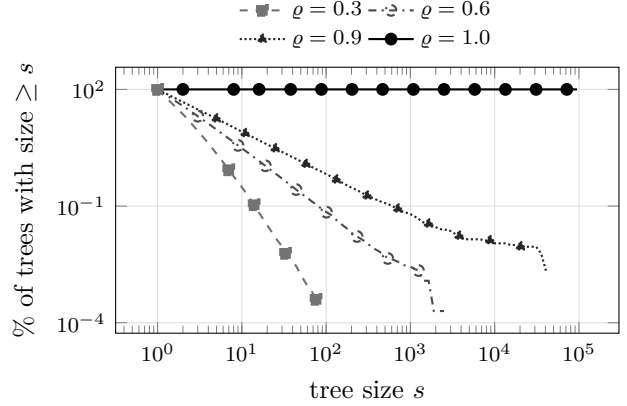


Figure 3: Distribution of the tree size: for each size s , the percentage of trees with at least s vertices, one curve and one column per density; the table lists the same distribution at selected sizes, plus the largest tree observed. Instances: **mixed-forest**, $n = 100\,000$, 10 instances per density, 40 in total. Observations: the distribution is heavy-tailed at every density below 1: at $\varrho = 0.3$, 77% of the trees are single vertices and only 0.3% reach 10 vertices, yet the largest reaches 96. At $\varrho = 0.9$ the tail detaches completely – 30% of trees have ≥ 3 vertices and one tree of $\approx 4 \cdot 10^4$ vertices appears, four orders of magnitude above the median. This is the percolation-like transition that Fig. 14 sees as a timing effect: what changes between $\varrho = 0.6$ and $\varrho = 0.9$ is not the number of trees (it falls smoothly) but the emergence of one giant tree, and RaC’s per-component overhead is paid on the count of trees while its contraction rounds are amortized over the giant one.

family	#inst	median # closure layers	
		$n \leq 1\,000$	$n \geq 10\,000$
anti-correlated	400	342	30379
correlated	400	337	24825
exact-ties	400	105	7884
independent-positive	400	344	32374
independent-signed	400	346	33183
near-ties	400	164	1222

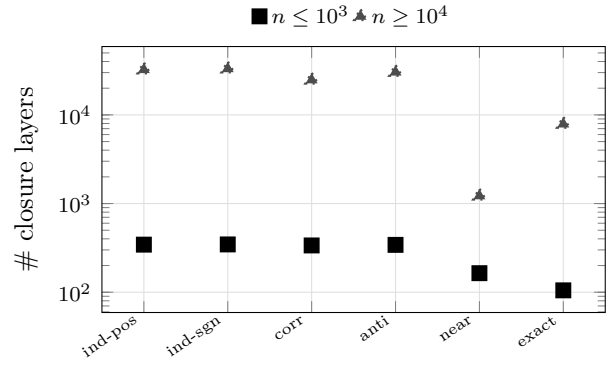


Figure 4: Median number of closure layers per coefficient family, reported separately for the small and the large sizes. Instances: **mixed-forest**, all four densities, $n \in \{100, \dots, 1\,000\}$ (2 400 instances) and $n \in \{10\,000, \dots, 100\,000\}$ (2 400 instances). Observations: how much parametric structure each family creates – the single most useful fact for reading absolute times, since the work of every algorithm scales with the number of breakpoints. At $n \geq 10^4$ the median layer count ranges from 1 222 (**near-ties**) to 33 183 (**independent-signed**), a factor of 27 at equal size. This is why the tie-stressing families are the *fastest* in every timing table below, which would otherwise look counter-intuitive.

n	#inst	# closure layers		
		median	min	max
100	240	57	10	91
200	240	106	24	177
300	240	158	35	267
400	240	205	53	350
500	240	265	66	438
600	240	314	80	521
700	240	366	95	611
800	240	418	107	709
900	240	470	124	782
1 000	240	528	134	873
10 000	240	5 032	436	8 449
20 000	240	9 826	558	16 728
30 000	240	14 718	645	24 852
40 000	240	19 434	674	32 779
50 000	240	24 226	741	40 675
60 000	240	28 663	787	48 463
70 000	240	32 662	825	55 961
80 000	240	36 690	864	63 666
90 000	240	40 251	888	71 224
100 000	240	43 390	915	78 393

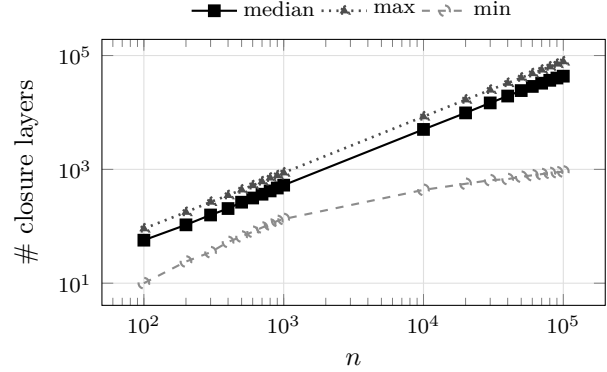


Figure 5: Median, minimum and maximum number of closure layers, per size. Instances: **mixed-forest**, n from 100 to 100 000, all four densities and six families, 240 instances per size. Observations: the median layer count grows essentially linearly in n , settling at 0.43 layers per vertex (43 390 layers at $n = 10^5$): a 10^5 -vertex instance typically has of the order of $4 \cdot 10^4$ breakpoints to enumerate. The min-max band, however, spans a factor of 86 at that size (915 to 78 393), and the reason is not noise but the coefficient families: **near-ties** produces $27\times$ fewer layers than the independent families (Fig. 4). Two instances of equal size can therefore carry very different amounts of parametric structure, which is the dominant source of instance-to-instance variation in every timing table below.

Table 2: Number of instances benchmarked, of timed runs, and of cross-algorithm disagreements, per instance class. Every group aggregates all sizes, densities, coefficient families and seeds of that class. The BPPF comparison is listed apart, BPPF being an external baseline rather than one of our algorithms. Zero disagreements everywhere: on each instance the algorithms returned the same partition, the same thresholds and the same canonical order.

instance class	n	#inst	timed runs	disagreements
mixed-forest	100–1 000	2 400	132 000	0
mixed-forest	10^4 – 10^5	2 400	24 000	0
path-mixed	100– 10^5	600	7 200	0
binary-mixed	100– 10^5	600	7 200	0
star-mixed	100– 10^5	600	6 480	0
in-forest	100– 10^5	4 800	57 600	0
out-forest	100– 10^5	4 800	57 600	0
total		16 200	292 080	0

group	#inst	layers	median	mean	max	singletons (%)
anti-correlated	10	60402	1	1.66	5029	73.1
correlated	10	56228	1	1.78	4997	68.2
exact-ties	10	16072	3	6.22	4960	0.0
independent-positive	10	61107	1	1.64	5065	74.4
independent-signed	10	61520	1	1.63	5077	75.3
near-ties	10	5903	7	16.94	4930	0.0
gen, $\varrho = 0.3$	6	33866	1	1.77	563	78.3
gen, $\varrho = 0.6$	6	29163	1	2.06	740	67.8
gen, $\varrho = 0.9$	6	23724	1	2.53	835	59.4
gen, $\varrho = 1.0$	6	22272	1	2.69	778	58.1
in, $\varrho = 0.3$	6	34065	1	1.76	522	78.8
in, $\varrho = 0.6$	6	29351	1	2.04	655	67.6
in, $\varrho = 0.9$	6	24232	1	2.48	803	58.2
in, $\varrho = 1.0$	6	22239	1	2.70	859	55.2
path	6	22885	2	2.62	1111	41.5
star	6	19435	1	3.09	5077	95.1

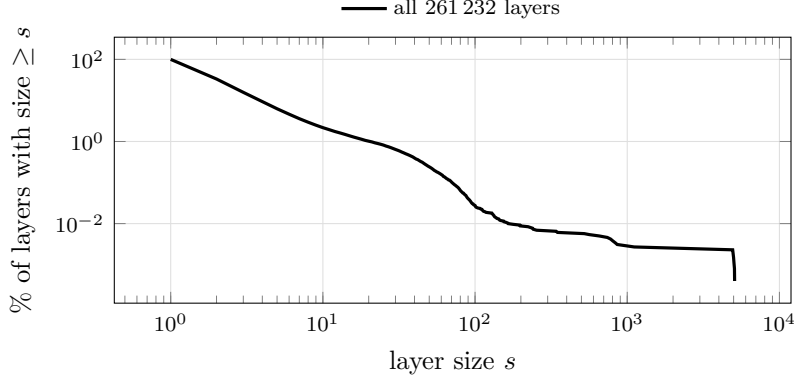


Figure 6: Distribution of the closure-layer *size* (number of vertices per layer): the table gives the number of layers, their median, mean and maximum size and the percentage of singleton layers per group; the plot gives the percentage of layers of size at least s . Instances: 60 instances at $n = 10\,000$ – all six coefficient families and four densities of **mixed-forest**, plus **star-mixed**, **path-mixed** and **in-forest** – for a total of 261 232 layers. Observations: *the distribution is extremely skewed*: overall median 1, mean 2.3, and two thirds of all layers are singletons – yet the largest single layer holds 5 077 vertices, half the graph. So a parametric solution is typically a long chain of one-vertex layers plus a handful of very large blocks, and an algorithm’s cost per layer matters more than its cost per vertex. Two group-level facts stand out. The tie-stressing families behave completely differently from the others: **near-ties** has median layer size 7 and *no* singletons at all (by construction, vertices share thresholds), against median 1 and 73–75% singletons for the independent families – which is the mechanism behind their much smaller layer counts (Fig. 4). And density compresses the largest layer by an order of magnitude: on sparse random forests the biggest layer holds 563 vertices, against $\approx 5\,000$ once the graph is a single tree, because a layer cannot straddle components.

- **E** restricts the orientation (in-forest, out-forest) to measure what a specialized algorithm gains from knowing it in advance.
- **G** is the external comparison against parametric pseudoflow. It sits on the same instances as B, so its numbers can be read against the rest of the study.

All campaigns were frozen in `docs/EXPERIMENTAL_PLAN_V3.md` before any run, together with the two size cutoffs, so that no scoping decision was taken after seeing results. Table 3 gives the definitions, Table 4 how much work each algorithm actually did, and Table 2 the correctness outcome per campaign.

Table 3: Campaign definitions. Every campaign uses all six coefficient families and ten seeds; the random topologies use all four densities. Two preregistered size cutoffs apply, in both cases because the asymptotic trend is already established below them and running further would consume hours without adding information: PaC/DPaC stop at $n = 20\,000$ on random forests, and HPaC/DHPaC stop at $n = 20\,000$ on stars (where PaC is cheap and runs the full range instead).

	Topology	Sizes n	Inst./size	Reps	Algorithms
A	all six	≤ 11 (exhaustive at 4)	–	–	all, against the enumeration oracle
B	mixed-forest	100, 200, ..., 1 000	240	11	PaC, DPaC, HPaC, DHPaC, RaC
C	mixed-forest	10 000, ..., 100 000	240	3	HPaC, DHPaC, RaC; PaC, DPaC at 10^4 , $2 \cdot 10^4$
D	path-, binary-, star-mixed	10 sizes, 100 ... 100 000	60	3	HPaC, RaC, DHPaC, PaC, DPaC
E	in-, out-forest	20 sizes, 100 ... 100 000	240	3	HPaC, DHPaC, HIPaC/HOPaC, RaC
G	mixed-forest	100, 200, ..., 1 000	240	3	HPaC against BPPF

Table 4: Work actually done per algorithm over the sweep. The runs-per-instance column reflects the mix of repetition counts and cutoffs: PaC and DPaC appear on fewer instances (size cutoffs) but with more repetitions each, since they were mostly exercised on the medium campaign where 11 repetitions were used. RaC appears on the most instances, being the only algorithm run on every class at every size.

algorithm	instances	timed runs	runs/instance
PaC	4 080	31 200	7.6
DPaC	3 480	29 400	8.4
HPaC	16 080	67 440	4.2
DHPaC	16 080	67 440	4.2
HIPaC	4 800	14 400	3.0
HOPaC	4 800	14 400	3.0
RaC	16 200	67 800	4.2
total	16 200	292 080	

8 Random forests

The two random campaigns answer two different questions. At medium sizes the question is whether the heap is worth its maintenance cost (Section 8.1); at large sizes it is how the three surviving algorithms scale, and how much the answer depends on the density of the forest (Section 8.2).

8.1 Medium sizes: direct scan versus heap

Campaign B runs all five of our algorithms on the same 2 400 instances, which makes it the one place where the primal/dual and scan/heap choices can be compared without any confounding factor. It

is also the size range where the heap’s overhead is still comparable to its benefit, so it is where the crossover can be located.

n	#inst	median CPU time (ms)				
		PaC	DPaC	HPaC	DHPaC	RaC
100	240	0.1	0.1	0.1	0.1	0.3
200	240	0.2	0.2	0.2	0.2	0.6
300	240	0.3	0.3	0.3	0.3	0.9
400	240	0.5	0.5	0.3	0.3	1.2
500	240	0.7	0.7	0.4	0.4	1.4
600	240	0.9	1.0	0.5	0.5	1.8
700	240	1.2	1.3	0.6	0.6	2.0
800	240	1.8	1.6	0.8	0.7	2.7
900	240	2.2	2.0	0.9	0.8	3.3
1 000	240	2.2	2.4	0.9	0.9	3.0

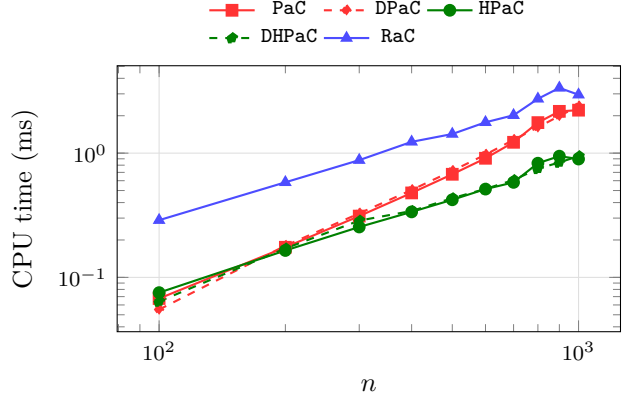


Figure 7: Median CPU time in milliseconds of the five algorithms, per size. Instances: **mixed-forest**, $n \in \{100, 200, \dots, 1\,000\}$, 240 instances per size (four densities $\times$ six coefficient families $\times$ ten seeds), 11 repetitions each. Observations: the scan-based and the heap-based algorithms are indistinguishable up to $n \approx 300$ and the heap pays off from $n \approx 400$ on; each dual tracks its primal over the whole range, which is why the manuscript reports only the primals. The mechanism is the candidate selection: PaC re-scans every candidate ratio at every iteration, while HPaC extracts the maximum in logarithmic time and refreshes only the candidates touched by the last peel or contraction.

n	#inst	median PaC/HPaC	IQR
100	240	0.910	0.609
200	240	1.06	0.747
300	240	1.19	0.879
400	240	1.31	1.06
500	240	1.45	1.27
600	240	1.65	1.37
700	240	1.79	1.49
800	240	1.94	1.72
900	240	2.06	1.96
1 000	240	2.29	2.13

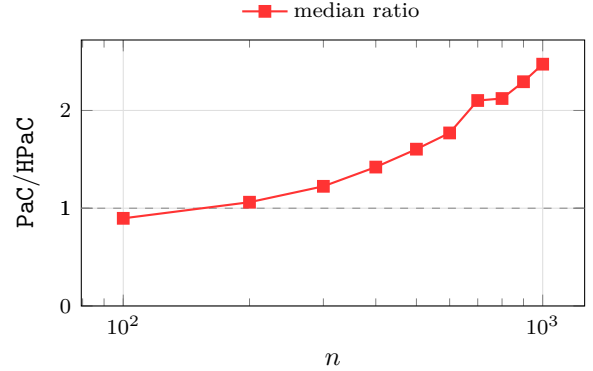


Figure 8: Median of the paired per-instance ratio PaC/HPaC, per size, with the number of paired instances and the IQR of the ratio. Instances: **mixed-forest**, $n \in \{100, 200, \dots, 1\,000\}$, 240 instances per size. Observations: the ratio is 0.91 at $n = 100$ – the scan is marginally faster where heap maintenance does not yet pay for itself – crosses 1 near $n = 400$ and reaches 2.3 at $n = 1\,000$. Beyond this range it keeps growing, 20.3 at $n = 10^4$ and 42.9 at $n = 2 \cdot 10^4$ (Section 8.2), which is the practical meaning of the $\mathcal{O}(n^2)$ bound against $\mathcal{O}(n \log n)$ behaviour in practice. The dashed line marks parity.

8.2 Large sizes, and the effect of density

Campaign C drops the two direct-scan algorithms beyond $n = 20\,000$ (their quadratic growth is established by then) and asks the question that matters at scale: how do HPaC, its dual and RaC compare on forests with up to 10^5 vertices, and how much of the answer depends on the density of the forest. As it turns out, more than it depends on the size.

ϱ	#inst	median CPU time (ms)		
		PaC	HPaC	RaC
0.3	600	0.6	0.2	0.6
0.6	600	0.7	0.3	1.2
0.9	600	0.8	0.7	2.1
1.0	600	0.9	0.9	2.3

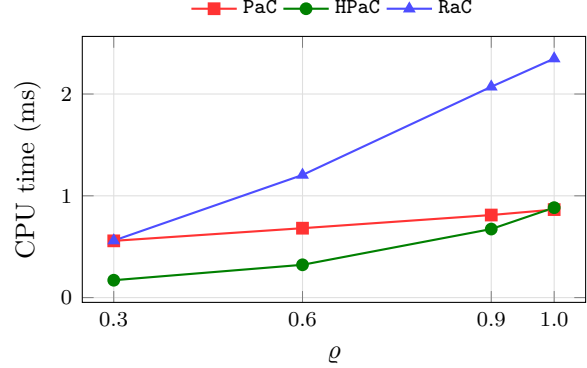


Figure 9: Median CPU time in milliseconds of PaC, HPaC and RaC, per arc density. Instances: *mixed-forest*, $n \in \{100, \dots, 1000\}$ pooled, 600 instances per density. Observations: every algorithm slows down as the forest becomes more connected – more arcs mean more closure sums to propagate – but by very different factors from $\varrho = 0.3$ to $\varrho = 1.0$: $4.5\times$ for HPaC (0.2 to 0.9 ms), $3.8\times$ for RaC (0.6 to 2.3 ms) and only $1.5\times$ for PaC (0.6 to 0.9 ms), which maintains nothing and therefore has nothing to refresh when the forest gains arcs. HPaC overtakes PaC on sparse forests and merely draws level with it on single spanning trees: the heap’s advantage at these sizes comes from the sparse end. This is the medium-size beginning of the density effect that dominates Section 8.2.

family	#inst	median CPU time (ms)		
		PaC	HPaC	RaC
anti-correlated	400	0.8	0.4	1.2
correlated	400	0.8	0.4	1.2
exact-ties	400	0.5	0.3	1.2
independent-positive	400	0.8	0.4	1.3
independent-signed	400	0.8	0.4	1.2
near-ties	400	0.6	0.3	1.2

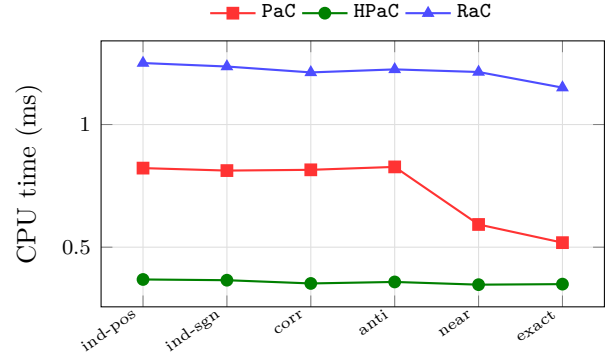


Figure 10: Median CPU time in milliseconds of PaC, HPaC and RaC, per coefficient family. Instances: *mixed-forest*, $n \in \{100, \dots, 1000\}$ and all four densities pooled, 400 instances per family. Observations: the two tie-stressing families are the fastest for PaC (0.5–0.6 against 0.8 ms) and for HPaC (0.3 against 0.4 ms), consistently with their smaller layer counts (Fig. 4) – fewer breakpoints, fewer iterations – while for RaC the six families all sit at 1.2–1.3 ms, indistinguishable at this resolution. The ordering between algorithms is the same in all six families, so no family reverses a conclusion.

family	#inst	median RaC/HPaC	IQR
anti-correlated	400	3.12	0.674
correlated	400	3.17	0.684
exact-ties	400	3.30	0.724
independent-positive	400	3.19	0.633
independent-signed	400	3.18	0.636
near-ties	400	3.17	0.723

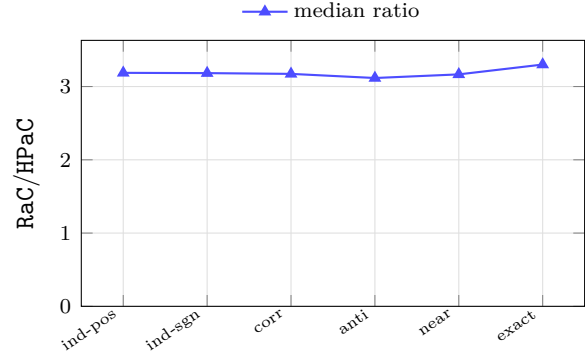


Figure 11: Median of the paired per-instance ratio RaC/HPaC, per coefficient family, with the number of paired instances and the IQR. Instances: **mixed-forest**, $n \in \{100, \dots, 1\,000\}$ and all four densities pooled, 400 instances per family. Observations: the ratio is flat at 3.1–3.3 across all six families – at these sizes the coefficient family changes how much work there is, not which algorithm does it faster.

n	#inst	median CPU time (ms)		
		HPaC	DHPaC	RaC
10 000	240	12.7	13.2	50.5
20 000	240	28.0	29.8	131.5
30 000	240	44.9	47.2	262.8
40 000	240	62.9	66.9	399.7
50 000	240	84.3	92.3	543.9
60 000	240	96.1	117.9	652.5
70 000	240	117.4	141.7	837.0
80 000	240	141.1	161.5	1 027.5
90 000	240	164.9	198.1	1 304.7
100 000	240	203.9	214.6	1 709.4

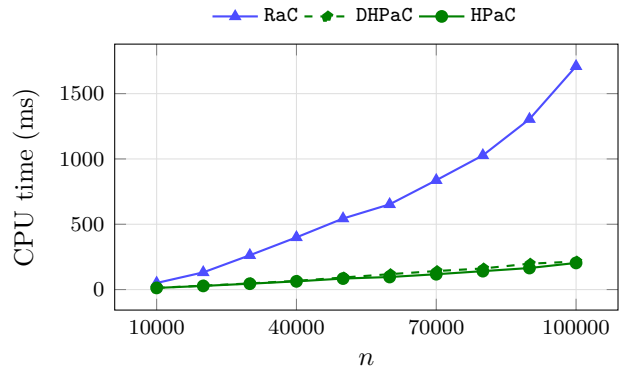


Figure 12: Median CPU time in milliseconds of HPaC, DHPaC and RaC, per size. Instances: **mixed-forest**, $n \in \{10\,000, 20\,000, \dots, 100\,000\}$, 240 instances per size (four densities $\times$ six families $\times$ ten seeds), 3 repetitions each. Observations: HPaC computes the full parametric solution on 10^5 vertices in about 0.2s (203.9ms). The paired ratio DHPaC/HPaC stays between 0.96 and 1.24 over the range, so the primal/dual choice is immaterial at scale – the difference is small enough that the median columns of the table and the paired ratio need not rank the two the same way at $n = 10^5$. RaC is above both at every size, by a factor that grows with n (Fig. 13).

n	#inst	median RaC/HPaC	IQR
10 000	240	4.18	2.34
20 000	240	5.10	4.29
30 000	240	6.13	6.40
40 000	240	7.10	8.02
50 000	240	7.86	9.82
60 000	240	9.21	13.01
70 000	240	10.37	15.36
80 000	240	11.29	16.89
90 000	240	12.47	18.37
100 000	240	13.26	19.45

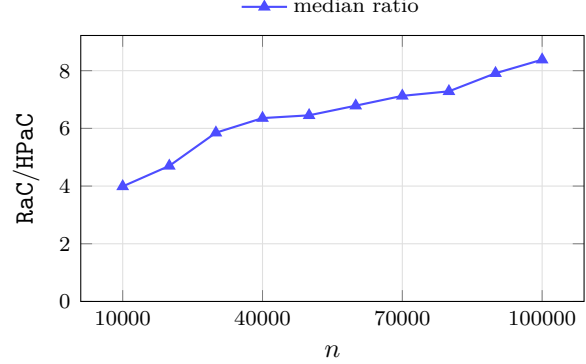


Figure 13: Median of the paired per-instance ratio RaC/HPaC , per size, with the number of paired instances and the IQR. Instances: **mixed-forest**, $n \in \{10\,000, \dots, 100\,000\}$, 240 instances per size. Observations: the ratio grows from 4.2 at $n = 10^4$ to 13.3 at $n = 10^5$, and its IQR widens with n as well, so RaC 's disadvantage grows *and* becomes more variable. The variability is not noise: it is the mixture of the four densities, which Fig. 14 separates.

ϱ	#inst	median RaC/HPaC	IQR
0.3	600	12.92	10.24
0.6	600	17.03	12.52
0.9	600	4.97	1.89
1.0	600	2.00	0.359

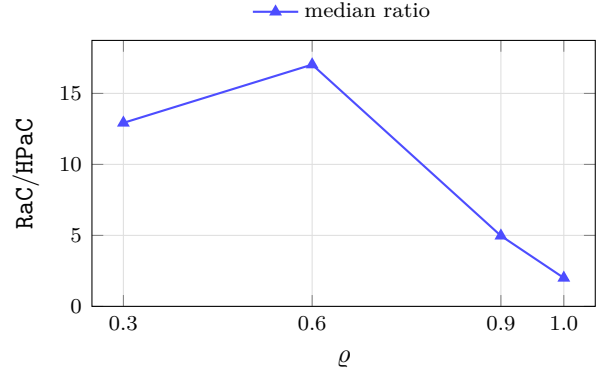


Figure 14: Median of the paired per-instance ratio RaC/HPaC , per arc density, with the number of paired instances and the IQR. Instances: **mixed-forest**, $n \in \{10\,000, \dots, 100\,000\}$ pooled, 600 instances per density. Observations: the ratio is 12.9 at $\varrho = 0.3$, 17.0 at $\varrho = 0.6$, 5.0 at $\varrho = 0.9$ and only 2.0 on single spanning trees, with the IQR shrinking from 10.2 to 0.4 in the same direction. A sparse forest splits into tens of thousands of tiny trees (Fig. 2), on which HPaC 's contractions are almost free while RaC still pays cluster bookkeeping per tree; on one spanning tree the two are closest. The ordering never reverses, but the margin varies by almost an order of magnitude – the population is a mixture of four regimes, which is exactly what the widening IQR of Fig. 13 is made of, and the reason a single aggregate over densities would mislead.

ϱ	#inst	median CPU time (ms)		
		HPaC	DHPaC	RaC
0.3	600	27.8	34.4	349.2
0.6	600	52.4	62.3	902.1
0.9	600	140.7	159.3	709.4
1.0	600	221.4	261.4	432.7

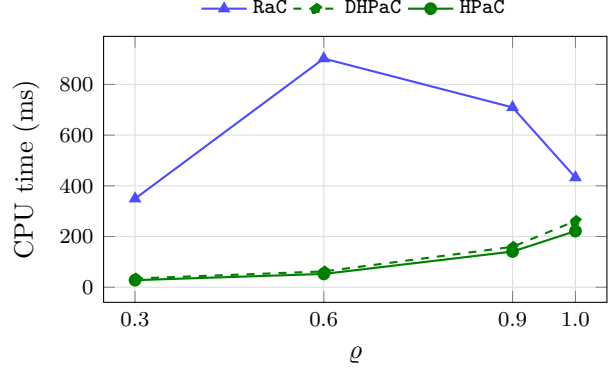


Figure 15: Median CPU time in milliseconds of RaC, DHPaC and HPaC, per arc density. Instances: *mixed-forest*, $n \in \{10\,000, \dots, 100\,000\}$ pooled, 600 instances per density. Observations: this is Fig. 14 in absolute terms, and the two algorithms respond to density in opposite ways. HPaC grows monotonically, by a factor 8 from $\varrho = 0.3$ to $\varrho = 1.0$ (27.8 to 221.4 ms): more arcs mean more closure sums to propagate per move. RaC is *non-monotone* – it peaks at $\varrho = 0.6$ (902 ms) and falls to 433 ms on single spanning trees – because its cost is a per-tree overhead plus contraction work per tree: at $\varrho = 0.3$ there are $\approx 70\,000$ trees but 77% are isolated vertices and cost nothing to contract, at $\varrho = 0.6$ there are still 40 000 trees and most are non-trivial (the worst combination), and from $\varrho = 0.9$ on the count collapses and the work concentrates in one large tree, where the $\mathcal{O}(\log n)$ contraction rounds are amortized best (Figs. 2 and 3).

family	#inst	median CPU time (ms)		
		HPaC	DHPaC	RaC
anti-correlated	400	68.5	78.2	563.3
correlated	400	67.2	75.6	543.8
exact-ties	400	63.0	71.2	505.1
independent-positive	400	70.7	79.2	559.9
independent-signed	400	70.2	77.8	552.1
near-ties	400	62.0	68.0	490.3

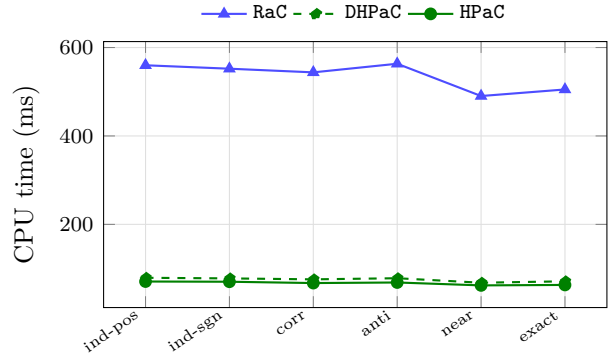


Figure 16: Median CPU time in milliseconds of RaC, DHPaC and HPaC, per coefficient family. Instances: *mixed-forest*, $n \in \{10\,000, \dots, 100\,000\}$ and all four densities pooled, 400 instances per family. Observations: the spread across the six families is 14% for HPaC (62.0–70.7 ms), 16% for DHPaC and 15% for RaC (490–563 ms) – an order of magnitude less than the spread across densities in Fig. 15, and in the same direction for all three algorithms, **near-ties** and **exact-ties** being the fastest for each of them.

family	#inst	median RaC/HPaC	IQR
anti-correlated	400	6.00	11.17
correlated	400	6.02	11.49
exact-ties	400	5.98	13.35
independent-positive	400	5.80	11.05
independent-signed	400	5.90	10.99
near-ties	400	6.11	13.38

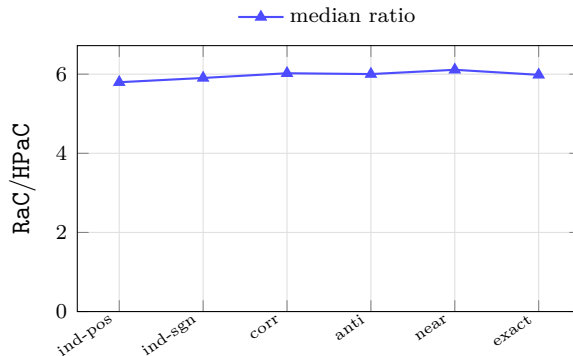


Figure 17: Median of the paired per-instance ratio RaC/HPaC, per coefficient family, with the number of paired instances and the IQR. Instances: *mixed-forest*, $n \in \{10\,000, \dots, 100\,000\}$ and all four densities pooled, 400 instances per family. Observations: the ratio lies between 5.80 and 6.11 in all six families, a 5% spread. Read against Fig. 14, where the same ratio moves between 2.0 and 17.0, this is the clearest single statement of the study: the topology decides which algorithm wins, the coefficient family only how much work there is to do.

8.3 What the random campaigns say

Three things follow from Figs. 7, 8, 12 to 14 and 17 and are worth stating explicitly, because none of them is visible from the complexity bounds alone.

- **The heap is worth it, but only past a size threshold.** Below $n \approx 300$ the direct scan is as good and at $n = 100$ marginally better: the heap’s maintenance is not amortized yet. The threshold is small enough that it does not matter in practice, but it does mean that for tiny instances the simpler algorithm is the right choice.
- **Density is a first-order factor, and it acts against RaC.** The RaC/HPaC ratio moves from 2.0 to 17.0 as ϱ goes from 1.0 to 0.6 – a bigger swing than going from $n = 10^4$ to $n = 10^5$ at fixed density. Anyone benchmarking these algorithms on a single density would draw a substantially wrong conclusion about the margin. The reason is structural: RaC pays cluster bookkeeping per component, and a sparse forest is thousands of tiny components.
- **The coefficient family is a second-order factor.** It changes absolute times by at most 14% and the paired ratio by less than 6%, even though it changes the number of closure layers by a factor of 27 (Fig. 4). Time per layer, not time per vertex, is what the family moves – and it moves it for all algorithms together.

9 Structured classes

Paths, balanced binary trees and stars share one property that the random forests do not: the topology is fixed, so only the orientations and the coefficients vary. They therefore isolate the effect of the shape, and they are the classes where the theoretical complexity separation becomes visible. Paths and binary trees confirm the random-forest picture; stars invert it.

9.1 What the structured classes say

- **Paths and binary trees add no new regime.** The ratios sit where the random forests put them (5.8–9.0 and 3.2–3.9) and are flat in n over three orders of magnitude: on these shapes the asymptotic advantage of RaC never materializes within the sizes we can run.

n	#inst	median CPU time (ms)			
		PaC	HPaC	DHPaC	RaC
100	60	0.1	0.1	0.1	0.5
200	60	0.2	0.2	0.2	1.1
500	60	0.8	0.4	0.4	2.9
1000	60	2.9	0.8	0.9	6.1
2000	60	10.5	1.7	1.8	12.9
5000	60	–	4.4	4.7	37.9
10000	60	–	9.0	9.7	80.5
20000	60	–	19.2	20.9	175.3
50000	60	–	54.9	60.6	475.8
100000	60	–	117.6	134.0	926.2

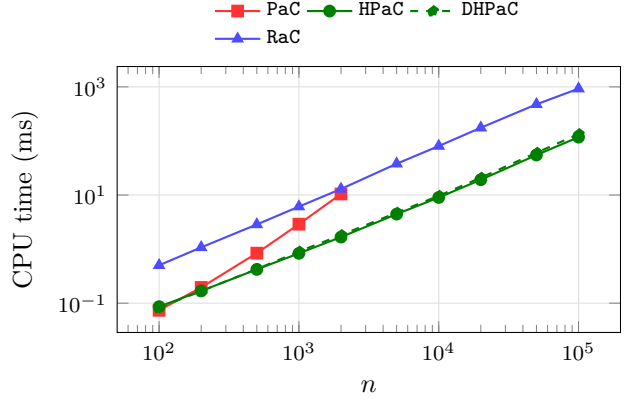


Figure 18: Median CPU time in milliseconds of PaC, HPaC, DHPaC and RaC, per size. Instances: **path-mixed** (one path, random orientations), n from 100 to 100 000, 60 instances per size (six coefficient families $\times$ ten seeds); PaC up to $n = 2000$, its preregistered cutoff. Observations: paths behave like random forests – median RaC/HPaC between 5.8 and 9.0 over the whole range, flat in n , and PaC left far behind by $n = 10^4$ as its $\mathcal{O}(n^2)$ bound predicts. A path gives RaC’s compress rounds no branching to exploit while every vertex has degree ≤ 2 , so no heap update ever cascades: the shape favours the heap.

family	#inst	median CPU time (ms)		
		PaC	HPaC	RaC
anti-correlated	100	0.9	3.1	25.8
correlated	100	0.9	3.1	25.3
exact-ties	100	0.7	2.9	22.3
independent-positive	100	0.9	3.1	25.8
independent-signed	100	0.8	3.0	25.5
near-ties	100	0.7	2.9	21.2

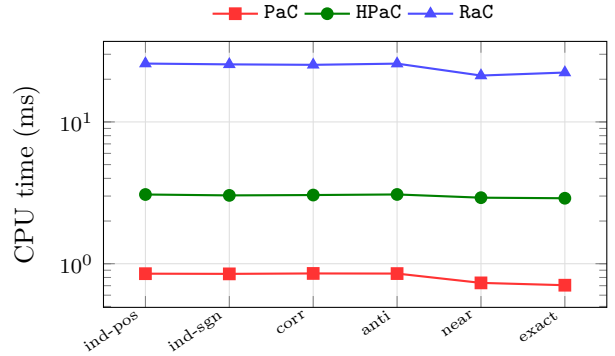


Figure 19: Median CPU time in milliseconds of PaC, HPaC and RaC, per coefficient family. Instances: **path-mixed**, all sizes pooled (n up to 2000 for PaC, up to 100 000 for the others), 100 instances per family for HPaC and RaC. Observations: the family changes absolute times by 22% for RaC (21.2–25.8 ms) and 7% for HPaC (2.9–3.1 ms), with **near-ties** and **exact-ties** the fastest as everywhere else; the paired RaC/HPaC ratio stays within 6.7–8.1 across the six families, so on paths too the family moves the amount of work and not the ranking. Absolute times pool sizes and are therefore comparable only across families, not against Fig. 18.

n	#inst	median CPU time (ms)			
		PaC	HPaC	DHPaC	RaC
100	60	0.1	0.1	0.1	0.4
200	60	0.2	0.2	0.2	0.9
500	60	0.9	0.6	0.7	2.3
1 000	60	3.1	1.3	1.4	4.9
2 000	60	11.2	2.6	2.7	10.1
5 000	60	–	6.8	7.2	27.3
10 000	60	–	14.3	15.0	57.5
20 000	60	–	32.0	32.1	123.1
50 000	60	–	92.3	92.5	315.6
100 000	60	–	205.6	211.1	663.7

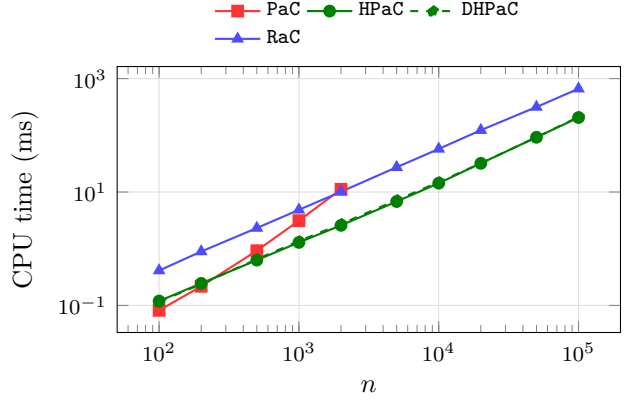


Figure 20: Median CPU time in milliseconds of PaC, HPaC, DHPaC and RaC, per size. Instances: **binary-mixed** (balanced binary tree, random orientations), n from 100 to 100 000, 60 instances per size; PaC up to $n = 2000$. Observations: the median RaC/HPaC ratio is between 3.2 and 3.9 over the whole range – lower than the 5.8–9.0 of paths, since a balanced tree gives RaC’s rake and compress rounds more to work with at each round, while for the heap-based schedules a bounded degree keeps updates cheap on both shapes.

family	#inst	median CPU time (ms)		
		PaC	HPaC	RaC
anti-correlated	100	0.9	4.7	18.9
correlated	100	0.9	4.6	18.7
exact-ties	100	0.7	4.4	16.0
independent-positive	100	0.9	4.7	18.7
independent-signed	100	0.9	4.7	18.7
near-ties	100	0.8	4.5	16.0

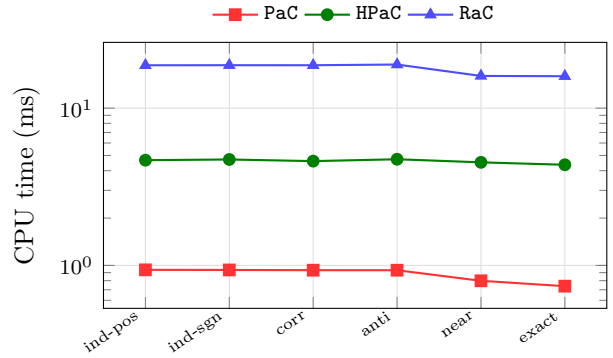


Figure 21: Median CPU time in milliseconds of PaC, HPaC and RaC, per coefficient family. Instances: **binary-mixed**, all sizes pooled (n up to 2 000 for PaC, up to 100 000 for the others), 100 instances per family for HPaC and RaC. Observations: the same flat picture as on paths, with a narrower band – 16.0 to 18.9 ms for RaC and 4.4 to 4.7 ms for HPaC, i.e. 18% and 7% of spread – and the paired RaC/HPaC ratio between 3.4 and 3.8 across the six families. The two tie-stressing families are again the fastest, and by the same relative amount for both algorithms, which is what “the family sets the amount of work” means quantitatively.

n	#inst	median CPU time (ms)			
		PaC	HPaC	DHPaC	RaC
100	60	0.1	0.6	0.6	0.4
200	60	0.2	2.0	2.0	0.8
500	60	1.1	12.6	13.2	2.1
1000	60	4.5	62.0	62.0	5.2
2000	60	17.6	231.2	226.1	9.7
5000	60	123.7	1427.5	1429.4	24.7
10000	60	643.1	6898.7	6479.8	63.3
20000	60	2890.8	26127.1	24619.1	122.1
50000	60	19336.1	–	–	291.7
100000	60	77541.1	–	–	606.8

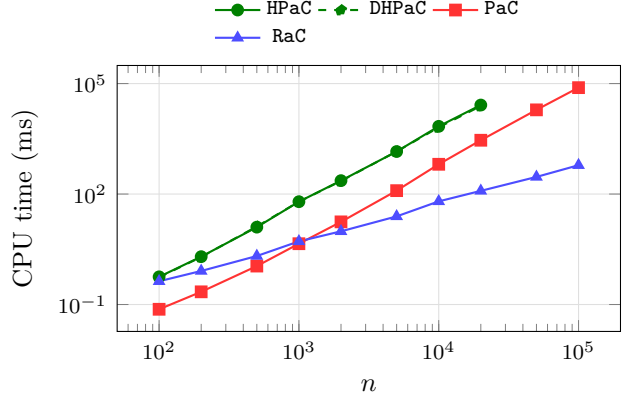


Figure 22: Median CPU time in milliseconds of HPaC, DHPaC, PaC and RaC, per size. Instances: **star-mixed** (one hub, $n - 1$ leaves, random orientations), n from 100 to 100 000, 60 instances per size; the heap-based algorithms up to $n = 20\,000$, PaC and RaC over the full range. Observations: this is the class that inverts the ordering. RaC is already faster at $n = 100$ and $\approx 250\times$ faster at $n = 20\,000$. Every peel or contraction touches the hub and invalidates the candidate ratios of a constant fraction of the remaining leaves, so *any* heap-based schedule pays $\Theta(n)$ maintenance per event, while RaC’s rake operations absorb all leaves in $\mathcal{O}(\log n)$ rounds regardless of the hub’s update history. PaC, which maintains no candidate structure at all, stays an order of magnitude below HPaC – on this class the schedule that maintains nothing beats the one that maintains a heap.

family	#inst	median CPU time (ms)		
		PaC	HPaC	RaC
anti-correlated	100	69.9	151.0	18.1
correlated	100	70.0	144.5	17.6
exact-ties	100	48.1	105.7	11.6
independent-positive	100	70.8	148.2	17.3
independent-signed	100	70.4	142.4	17.2
near-ties	100	50.1	110.9	12.6

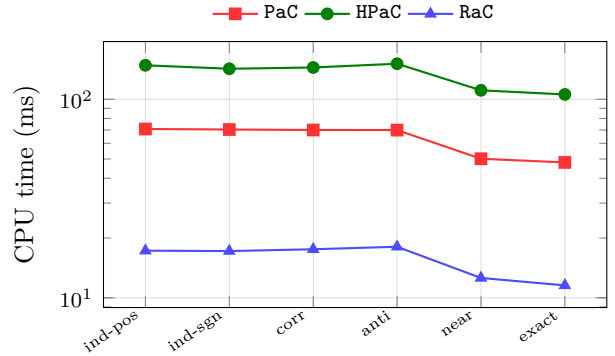


Figure 23: Median CPU time in milliseconds of PaC, HPaC and RaC, per coefficient family. Instances: **star-mixed**, all sizes pooled ($n \leq 20\,000$ for HPaC, up to 100 000 for PaC and RaC), 100 instances per family. Observations: the inversion of Fig. 22 is present in every family and its size barely moves – the paired RaC/HPaC ratio lies between 0.060 and 0.064 in all six – so the inversion is purely topological: no coefficient family produces it and none removes it. Note the vertical scale: HPaC is two orders of magnitude above RaC on this class, whereas on every other class it is below.

n	#inst	median RaC/PaC	IQR
100	60	5.78	0.454
200	60	3.68	0.192
500	60	1.88	0.120
1 000	60	1.15	0.053
2 000	60	0.524	0.034
5 000	60	0.198	0.016
10 000	60	0.100	0.005
20 000	60	0.043	0.004
50 000	60	0.015	0.001
100 000	60	0.007	0.000

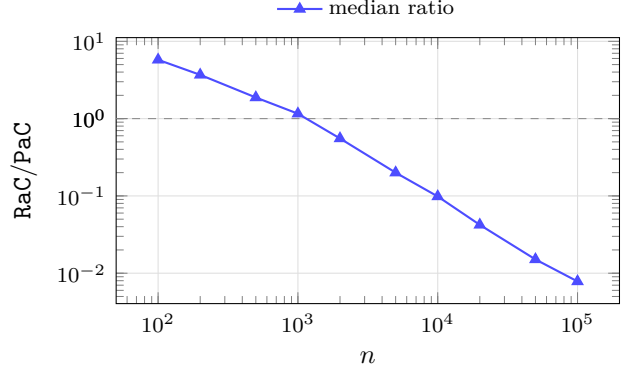


Figure 24: Median of the paired per-instance ratio RaC/PaC, per size, with the number of paired instances and the IQR. Instances: **star-mixed**, n from 100 to 100 000, 60 instances per size. Observations: this pairs the two algorithms that are *not* penalized by a candidate structure on a hub, and therefore isolates the asymptotic difference $\mathcal{O}(n \log n)$ against $\mathcal{O}(n^2)$ on the shape where it is visible. The crossover sits between $n = 1\,000$ and $n = 2\,000$ (1.15 then 0.52); at $n = 10^5$ the ratio is 0.007, i.e. RaC is $135\times$ faster, and the IQR is below 0.001, so the effect is uniform over instances and not driven by outliers. Dashed line marks parity.

- **Stars are the counterexample the theory predicts, and they are extreme.** The ratio moves by four orders of magnitude across the size range, from 0.75 to 0.004. This is the one place in the study where the complexity separation is not academic.
- **The star penalty is a property of the *schedule*, not of the algorithm.** PaC and HPaC implement the same algorithm; on stars the heap-based schedule is an order of magnitude slower than the scan-based one (Fig. 22), because maintaining candidate ratios is exactly what a hub makes expensive. A practitioner facing hub-shaped precedence graphs should reach for RaC, and failing that for the direct scan – not for the heap.

10 Single orientations

When the forest is known in advance to be a forest of in-trees or of out-trees, a specialized algorithm can exploit it. This section measures how much that knowledge is worth: it is a constant factor, not a change of regime, and the constant is remarkably stable across sizes, densities and coefficient families.

Note on the comparison with the first version of the manuscript. The specialized variants were reported there as $4\text{--}10\times$ faster than the general algorithm, against $1.3\text{--}1.9\times$ here. The direction is expected – the general HPaC is substantially faster than it was, see Section 11 – but the magnitude is only partly accounted for by the heap policy: comparing the two heap implementations directly on these instances gives $1.3\text{--}1.4\times$, not $3\text{--}4\times$. The remainder is attributable to the code being an independent rewrite rather than a refactoring, and to instances regenerated under different coefficient families. Absolute times across the two versions are therefore not comparable; only trends are.

10.1 What the oriented classes say

- **The specialization buys a constant, around $1.5\times$ to $2\times$ overall, and it does not erode with size.** The ratio at $n = 10^5$ is as good as at $n = 10^3$ (Figs. 26 and 28) and it is the same in all six coefficient families (Fig. 31).

n	#inst	median CPU time (ms)			
		HPaC	DHPaC	HIPaC	RaC
100	240	0.1	0.1	0.1	0.3
200	240	0.2	0.2	0.1	0.6
300	240	0.3	0.3	0.2	0.9
400	240	0.4	0.4	0.2	1.2
500	240	0.4	0.5	0.3	1.4
600	240	0.5	0.6	0.3	1.7
700	240	0.7	0.8	0.4	2.0
800	240	0.7	0.8	0.4	2.3
900	240	0.8	1.0	0.5	2.4
1 000	240	1.1	1.0	0.6	3.1
10 000	240	14.2	14.1	6.5	45.9
20 000	240	29.5	31.1	14.4	120.3
30 000	240	49.6	51.0	25.0	231.3
40 000	240	73.8	72.9	35.2	347.9
50 000	240	95.3	101.9	45.1	444.2
60 000	240	109.4	131.0	52.8	570.2
70 000	240	127.8	151.0	66.0	730.2
80 000	240	168.4	185.0	78.7	980.0
90 000	240	203.1	219.7	86.5	1 184.7
100 000	240	237.6	201.7	114.2	1 620.5

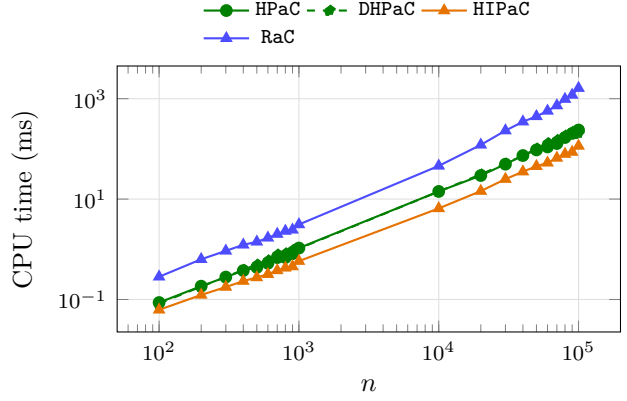


Figure 25: Median CPU time in milliseconds of HPaC, DHPaC, the specialized HIPaC and RaC, per size. Instances: **in-forest** (out-degree ≤ 1), $n \in \{100, \dots, 1000\} \cup \{10\,000, \dots, 100\,000\}$, 240 instances per size. Observations: HIPaC is below HPaC at every size and the two curves stay parallel, so the specialization is a constant factor and not a change of regime. The mechanism: on a single orientation the minimal preceding set of every arc collapses to a singleton, so one heap value per vertex suffices and no closure sum is ever propagated.

n	#inst	median HIPaC/HPaC	IQR
100	240	0.749	0.445
200	240	0.662	0.474
300	240	0.628	0.554
400	240	0.615	0.553
500	240	0.619	0.578
600	240	0.580	0.573
700	240	0.565	0.597
800	240	0.603	0.622
900	240	0.582	0.589
1 000	240	0.564	0.597
10 000	240	0.521	0.682
20 000	240	0.557	0.707
30 000	240	0.570	0.781
40 000	240	0.567	0.741
50 000	240	0.575	0.765
60 000	240	0.566	0.793
70 000	240	0.589	0.819
80 000	240	0.589	0.786
90 000	240	0.574	0.789
100 000	240	0.563	0.789

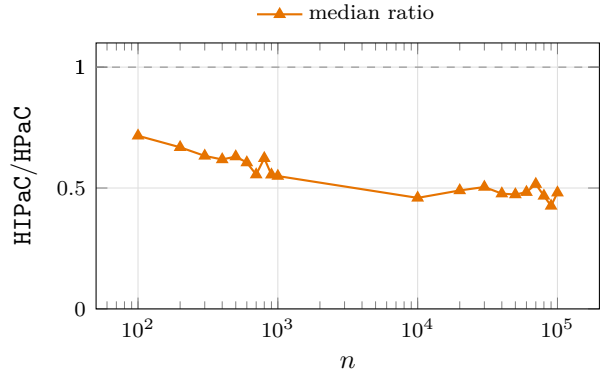


Figure 26: Median of the paired per-instance ratio HIPaC/HPaC, per size, with the number of paired instances and the IQR. Instances: **in-forest**, n from 100 to 100 000, 240 instances per size (all four densities and six families). Observations: the ratio lies between 0.52 and 0.75 with no erosion up to $n = 10^5$ – if anything the advantage grows mildly with n . The IQR is large (0.44 to 0.82) because these 240 instances mix four densities, on which the gain differs by a factor 6: Fig. 29 separates them. Dashed line marks parity.

n	#inst	median CPU time (ms)			
		HPaC	DHPaC	HOPaC	RaC
100	240	0.1	0.1	0.1	0.3
200	240	0.2	0.2	0.1	0.6
300	240	0.3	0.3	0.2	0.9
400	240	0.4	0.4	0.3	1.2
500	240	0.6	0.5	0.3	1.6
600	240	0.6	0.7	0.4	1.7
700	240	0.7	0.8	0.5	2.4
800	240	0.8	0.9	0.5	2.3
900	240	0.9	1.0	0.6	2.5
1 000	240	1.1	1.3	0.7	3.1
10 000	240	13.7	15.3	7.6	44.1
20 000	240	31.9	33.7	16.5	118.0
30 000	240	51.4	53.4	27.8	217.4
40 000	240	73.6	79.8	40.3	323.3
50 000	240	95.3	100.6	54.7	463.8
60 000	240	106.1	135.1	61.5	541.2
70 000	240	135.6	155.2	75.5	722.2
80 000	240	164.4	197.6	88.4	941.6
90 000	240	185.7	220.6	106.0	1 187.5
100 000	240	229.5	287.2	132.1	1 608.8

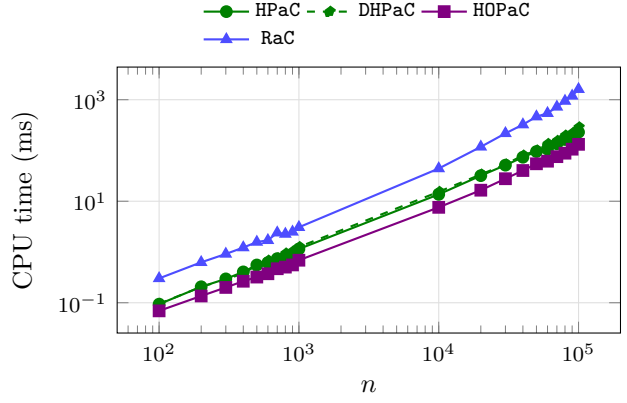


Figure 27: Median CPU time in milliseconds of HPaC, DHPaC, the specialized HOPaC and RaC, per size. Instances: **out-forest** (in-degree ≤ 1), n from 100 to 100 000, 240 instances per size. Observations: the picture mirrors the in-forest one of Fig. 25 at every size, as the symmetry of the two constructions predicts; the dual algorithm on the dual class buys the same constant factor, which is also a consistency check on the two implementations.

n	#inst	median HOPaC/HPaC	IQR
100	240	0.784	0.455
200	240	0.676	0.477
300	240	0.668	0.509
400	240	0.623	0.542
500	240	0.663	0.555
600	240	0.625	0.526
700	240	0.631	0.554
800	240	0.622	0.571
900	240	0.670	0.563
1 000	240	0.619	0.592
10 000	240	0.574	0.623
20 000	240	0.585	0.648
30 000	240	0.612	0.690
40 000	240	0.619	0.698
50 000	240	0.655	0.690
60 000	240	0.624	0.742
70 000	240	0.633	0.705
80 000	240	0.636	0.669
90 000	240	0.632	0.694
100 000	240	0.632	0.682

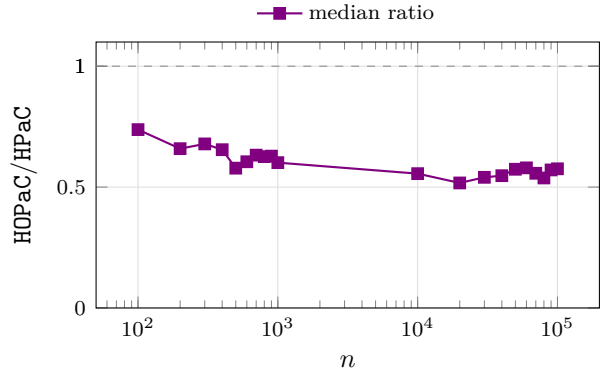


Figure 28: Median of the paired per-instance ratio HOPaC/HPaC, per size, with the number of paired instances and the IQR. Instances: **out-forest**, n from 100 to 100 000, 240 instances per size. Observations: the ratio lies between 0.57 and 0.78 and is never more than 0.09 from the in-forest value at the same size (Fig. 26) – the two specializations are worth the same constant, and neither erodes with n . Dashed line marks parity.

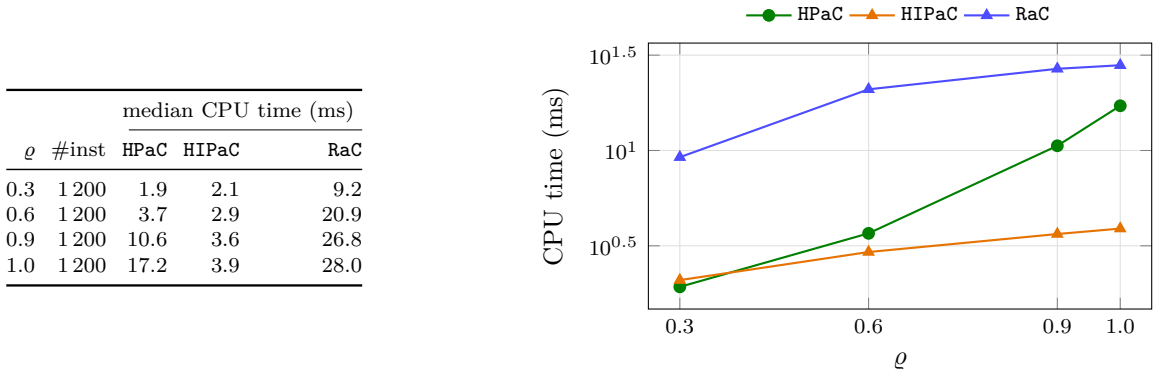


Figure 29: Median CPU time in milliseconds of HPaC, the specialized HIPaC and RaC, per arc density. Instances: **in-forest**, all twenty sizes pooled, 1 200 instances per density. Observations: this is where the specialization’s gain actually comes from, and it is *not* uniform. From $\varrho = 0.3$ to $\varrho = 1.0$, HPaC slows down by a factor 9 (1.9 to 17.2ms) while HIPaC slows down by less than a factor 2 (2.1 to 3.9ms): the paired ratio HIPaC/HPaC therefore falls from 1.23 at $\varrho = 0.3$ – where the specialization is a small *loss* – to 0.84, 0.34 and 0.21 at $\varrho = 0.6, 0.9$ and 1.0 , a $4.7\times$ gain on single spanning in-trees. The reason is exactly the propagation the specialization removes: on a fragmented forest of mostly isolated vertices (Fig. 2) there is almost no closure sum to propagate, so HPaC pays nothing for its generality and HIPaC’s setup cost is not recovered; the more connected the forest, the more the general algorithm propagates. The single figure “0.52–0.75” of Fig. 26 is thus an average over four quite different regimes.

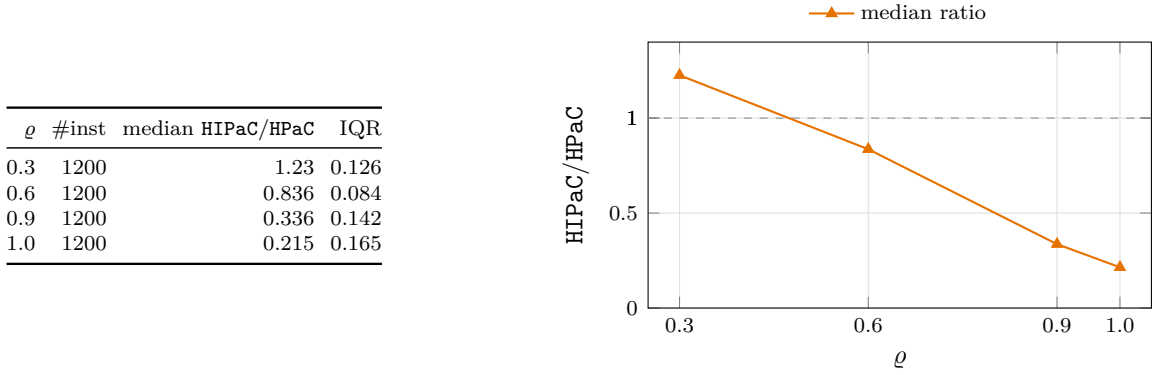


Figure 30: Median of the paired per-instance ratio HIPaC/HPaC, per arc density, with the number of paired instances and the IQR. Instances: **in-forest**, all twenty sizes pooled, 1 200 instances per density. Observations: the same effect as Fig. 29 in paired form, which is the form that admits no aggregation artefact – the ratio is computed on each instance and then aggregated. It crosses parity between $\varrho = 0.3$ and $\varrho = 0.6$, and the IQR stays below 0.17 throughout, so within one density the effect is tight: the wide IQR of Fig. 26 is entirely the mixture of densities. Practical reading: the specialized variants are worth having when the input is a connected in-tree or out-tree, and worth nothing on a heavily fragmented forest.

family	#inst	median HIPaC/HPaC	IQR
anti-correlated	800	0.689	0.705
correlated	800	0.697	0.741
exact-ties	800	0.671	0.617
independent-positive	800	0.676	0.710
independent-signed	800	0.652	0.659
near-ties	800	0.653	0.620

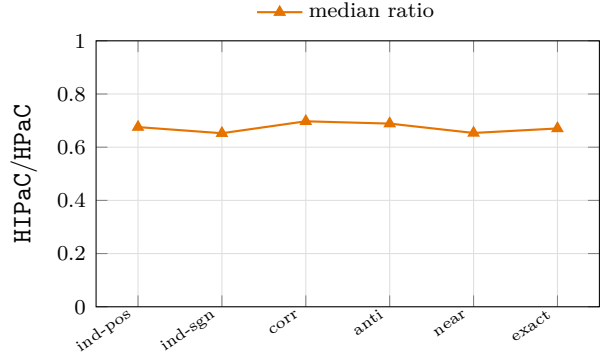


Figure 31: Median of the paired per-instance ratio HIPaC/HPaC, per coefficient family, with the number of paired instances and the IQR. Instances: *in-forest*, all sizes and densities pooled, 800 instances per family. Observations: the ratio lies between 0.65 and 0.70 in all six families, a 7% spread – against the factor 6 across densities of Fig. 30. There is no family in which the advantage of the specialization disappears, and none in which it is much larger: as everywhere else in this report, the coefficients set the amount of work and the topology sets who wins.

family	#inst	median RaC/HPaC	IQR
anti-correlated	800	3.01	2.17
correlated	800	3.07	2.30
exact-ties	800	3.24	2.43
independent-positive	800	3.02	2.27
independent-signed	800	3.04	2.26
near-ties	800	3.03	2.59

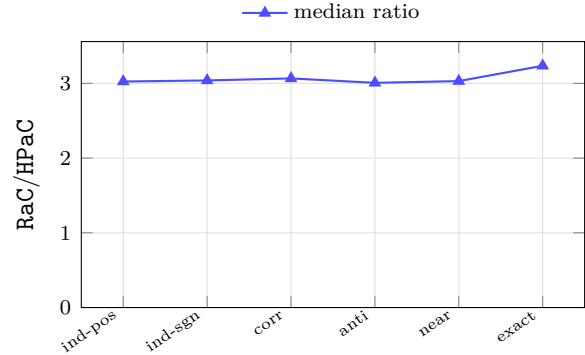


Figure 32: Median of the paired per-instance ratio RaC/HPaC, per coefficient family, with the number of paired instances and the IQR. Instances: *in-forest*, all sizes and densities pooled, 800 instances per family (out-forests give the same values within 6%). Observations: the ratio is 3.0–3.2 in every family, i.e. flat as on mixed forests, while by size it grows from about $2.8\times$ at $n \leq 1000$ to $12.8\times$ at $n = 10^5$ – so restricting the orientation does not change RaC’s standing against the heap-based algorithms, it only removes the need for closure-sum propagation from the latter.

- **But that constant is a strong function of the density, and this is the one place where the aggregate figure hides the effect.** The gain runs from 1.23 (a small loss) on forests with $\varrho = 0.3$ to 0.21 (a $4.7\times$ gain) on single spanning in-trees (Figs. 29 and 30). What the specialization removes is closure-sum propagation, and there is nothing to propagate in a forest of mostly isolated vertices: the more connected the input, the more the specialization is worth. Reported as a single number, the specialization looks like a modest constant; reported per density, it ranges from useless to a factor five.
- **It is smaller than the theory might suggest.** Both HIPaC and the general HPaC behave like $\mathcal{O}(n \log n)$ in practice on these instances, so what separates them is bookkeeping, not asymptotics: one heap value per vertex instead of two heaps plus closure-sum propagation.
- **Knowing the orientation is worth less than knowing the shape.** A factor 1.5–2 for a single orientation, against a factor 250 for a star. If one piece of structural information about the input can be exploited, the shape is the one that pays.

11 Implementation note: heap policy

The candidate heaps admit two implementations, indistinguishable in the asymptotic time bound and very different in practice.

- **Lazy deletion (push-only).** Every closure-sum change pushes a fresh entry; stale ones are dropped only when they surface at the top. Heap size is bounded by the number of touch operations, not by n . On a star this is pathological: every event touches the hub, each touch re-pushes one entry per incident arc, so the heap accumulates $\Theta(n^2)$ entries.
- **Periodic rebuild (used throughout this study).** Each heap is rebuilt from the live candidates whenever its size exceeds a constant multiple of the live candidate count: amortized $\mathcal{O}(\log n)$ per operation, worst-case time bound unchanged, memory $\mathcal{O}(n)$ on *every* input. This is what makes the implementation attain the space bound proved in the manuscript.

12 Comparison with parametric pseudoflow

BPPF evaluates minimum cuts at user-supplied parameter values, so the comparison must decide what to supply. We drive it in the setting most favourable to it, identical to the methodology of the manuscript’s first version: one BPPF process per instance receives, in BPPF’s native affine capacity format, the $k + 1$ values that bracket all k breakpoints – so it is spared the search for the breakpoints, and one call certifies the whole parametric solution. Timing is BPPF’s own internal solve timer (input parsing excluded) against HPaC’s in-process time on the same instance; fixed-point precision 10^{-6} ; all 2 400 instances with $n \leq 1\,000$.

Agreement and precision. The closures returned by the two methods agree at every probe on every instance, with one exception class: on **16 of the 2 400 instances** BPPF merges two consecutive closure layers whose thresholds differ by less than its fixed-point precision, and therefore does not recover the exact optimal layer sequence. All 16 belong to **near-ties** and **exact-ties**, the two families designed to place thresholds at exactly that distance – which is also why those families exist. Our algorithms are unaffected, comparing integers exactly.

Discarded variant. An earlier version of this comparison drove BPPF once per breakpoint, one process spawn per call. It is reported nowhere: the resulting ratio is dominated by process-creation overhead and says nothing about either algorithm.

n	#inst	lazy deletion		periodic rebuild	
		ms	MiB	ms	MiB
1 000	1	139.0	15.6	64.1	3.9
2 000	1	628.7	51.5	233.1	4.0
5 000	1	5 440.9	388.0	1 422.1	6.4
10 000	1	27 020.7	1 541.2	6 396.5	8.2

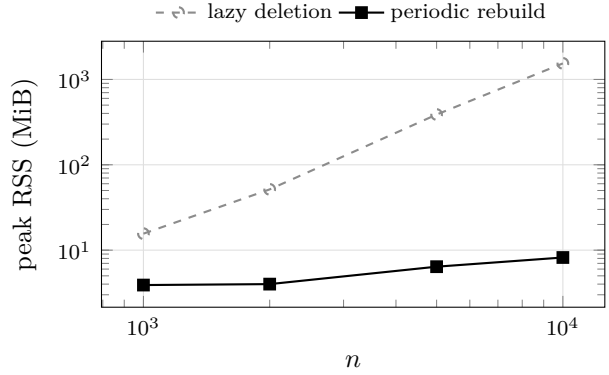


Figure 33: CPU time in milliseconds and peak RSS in MiB of HPaC under the two heap policies, per size; the plot shows the memory column of the table. Instances: **star-mixed**, **independent-positive**, seed 0, n from 1 000 to 10 000 – one instance per size, one repetition, each algorithm invoked in its own process, which is the only regime in which peak RSS is attributable to a single algorithm (Section 4). Observations: the lazy heap’s memory grows by a factor of 100 when n grows by 10 – quadratic, as predicted – reaching 1.5 GiB at $n = 10^4$ where the rebuild policy uses 8 MiB; extrapolating, the lazy policy would exhaust the 8 GiB ceiling by $n \approx 20\,000$. The rebuild policy is also $\approx 4\times$ faster here (and $\approx 2\times$ on large random forests), with bit-identical output on every instance tested, so the two policies are not a trade-off: one dominates. For reference, PaC, which maintains no candidate structure, peaks at 5.6 MiB at $n = 10^4$. The practical lesson: on this problem the choice of data structure inside one algorithm is worth as much as the choice between algorithms.

n	#inst	CPU time (ms)	
		HPaC	BPPF
100	240	0.1	0.1
200	240	0.1	0.3
300	240	0.2	0.6
400	240	0.3	1.0
500	240	0.4	1.5
600	240	0.5	2.1
700	240	0.5	2.8
800	240	0.7	3.6
900	240	0.8	4.5
1 000	240	0.9	5.5

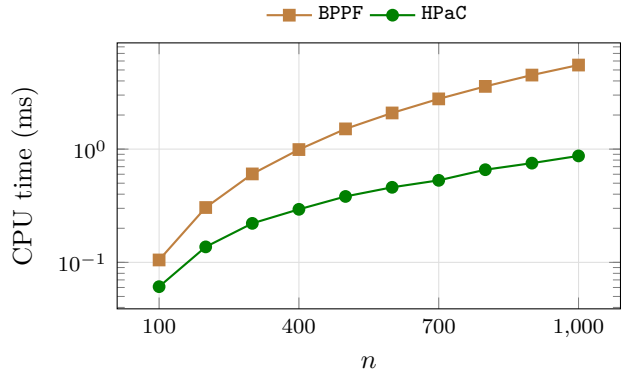


Figure 34: Median CPU time in milliseconds of HPaC and of BPPF, per size: HPaC’s in-process time for the full parametric sweep against BPPF’s own internal solve timer, input parsing excluded. Instances: **mixed-forest**, $n \in \{100, 200, \dots, 1\,000\}$, 240 instances per size, 2 400 in total; BPPF is given the $k + 1$ parameter values bracketing the k breakpoints, in its native affine capacity format, precision 10^{-6} . Observations: the median paired ratio BPPF/HPaC grows from 1.5 at $n = 100$ to 4.5 at $n = 1\,000$, overall median 3.3, even though the breakpoints are handed to BPPF for free. Scope: forests with $n \leq 1\,000$, whereas BPPF solves a strictly more general problem on arbitrary precedence graphs, so nothing here transfers outside this class – and recall that on a forest BPPF’s bound reads $\mathcal{O}(n^2 \log n)$ (Table 1), so the measured factor of a few is far kinder to it than the bounds. The times are sub-millisecond at the small end, which is why the ratio is computed per instance and not from these two columns.

13 Dispersion of the measurements

Every ratio reported above is a median over instances of a median over repetitions. This section reports the second-level dispersion – how much the repetitions of a single instance differ from each other – because that is the noise floor against which every claimed difference has to be read. It is expressed as the relative IQR (Section 3): the IQR over the repetitions of one instance, divided by that instance’s median time.

		median relative IQR of the timing (%)		
n	#inst	HPaC	DHPaC	RaC
10 000	240	0.5	2.2	4.6
20 000	240	0.6	2.3	3.5
30 000	240	0.9	2.2	2.8
40 000	240	1.0	2.1	2.1
50 000	240	1.3	2.2	1.5
60 000	240	1.7	1.7	1.5
70 000	240	1.6	1.6	1.3
80 000	240	1.5	1.7	1.1
90 000	240	1.7	2.2	1.0
100 000	240	1.8	2.3	1.2

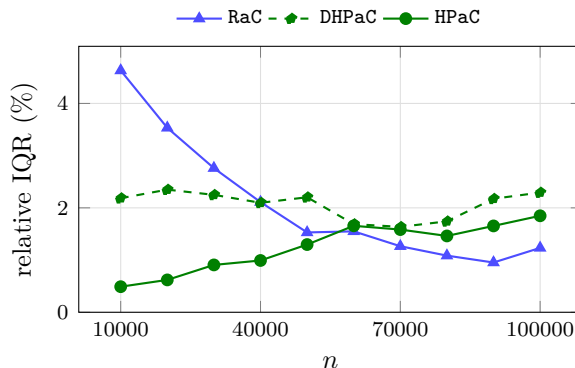


Figure 35: Median relative IQR of the timing, in percent, per size: for each instance, the IQR over its repetitions divided by its own median time, then the median of that over instances. Instances: **mixed-forest**, $n \in \{10\,000, \dots, 100\,000\}$, 240 instances per size, 3 repetitions each. Observations: 0.5% to 1.8% for HPaC, 1.6% to 2.3% for DHPaC, and 4.6% falling to 1.2% for RaC – the longer a run, the smaller its relative noise, which is why the largest sizes are the quietest. This is the number that says how much of a reported difference is signal: every ratio in this report is at least an order of magnitude larger than the measurement noise it rests on. The single exception is the near-tie 0.91 against 1 at $n = 100$ in Section 8.1, which is for that reason reported as a near-tie and not as an advantage.

A Per-family detail at every size

The full cross-tabulation behind every by-family statement above: median CPU time per coefficient family and per size, for the two headline algorithms, on both random campaigns. The family ordering is stable across sizes – the two tie-stressing families are consistently the fastest for both algorithms, by a margin that widens with n (at $n = 10^5$, **near-ties** costs HPaC about 12% less than **independent-positive**) – and no family ever reverses the relative standing of the two algorithms. The plots show four representative families to stay readable; the tables give all six.

B Reproducing this report

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
ctest --test-dir build --output-on-failure      # correctness gate
tools/run_night_sweep.sh                       # all campaigns, one command
tools/build_report.sh                           # aggregate + emit all tables
cd report && latexmk -pdf computational_report.tex
```

		median relative IQR of the timing (%)			
n	#inst	HPaC	HIPaC	RaC	
100	240	8.7	4.2	12.3	
200	240	4.5	3.1	4.4	
300	240	3.1	2.9	5.4	
400	240	3.7	3.2	4.7	
500	240	2.5	3.2	3.4	
600	240	3.0	2.4	3.1	
700	240	3.0	2.3	2.7	
800	240	3.4	2.4	3.0	
900	240	2.7	2.3	3.1	
1 000	240	3.9	2.2	4.1	
10 000	240	4.5	2.4	3.6	
20 000	240	3.6	2.9	2.7	
30 000	240	2.6	3.2	2.3	
40 000	240	2.6	3.3	1.6	
50 000	240	2.6	3.5	1.2	
60 000	240	2.6	3.4	1.1	
70 000	240	2.5	3.7	0.9	
80 000	240	2.7	3.2	0.7	
90 000	240	2.6	3.0	0.9	
100 000	240	2.5	2.6	1.1	

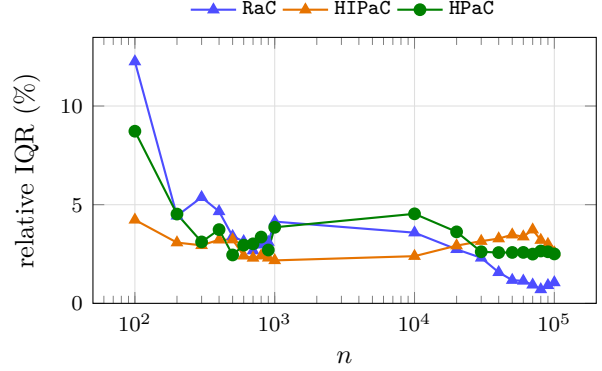


Figure 36: Median relative IQR of the timing, in percent, per size, defined as in Fig. 35. Instances: **in-forest**, n from 100 to 100 000, 240 instances per size, 3 repetitions each. Observations: 2.2%–4.2% for HIPaC against 2.5%–8.7% for the general HPaC, so the specialized variant is at least as stable as the algorithm it replaces and its advantage is not an artefact of variance; RaC goes from 12.3% at $n = 100$ down to 1.1% at $n = 10^5$. Noise is largest at $n = 100$, where a run lasts tens of microseconds and the timer resolution starts to show; from $n = 200$ on every algorithm stays below 6%, an order of magnitude below the 1.5 – $4.7\times$ effects measured in Section 10.

		median HPaC CPU time (ms) per family					
n	#inst	anti	corr	exact	ind-pos	ind-sgn	near
100	40	0.1	0.1	0.1	0.1	0.1	0.1
200	40	0.2	0.2	0.2	0.2	0.2	0.2
300	40	0.2	0.2	0.2	0.3	0.2	0.3
400	40	0.3	0.3	0.3	0.4	0.4	0.3
500	40	0.4	0.4	0.4	0.4	0.5	0.4
600	40	0.6	0.6	0.5	0.5	0.5	0.5
700	40	0.6	0.6	0.6	0.7	0.7	0.7
800	40	0.9	0.8	0.8	0.8	0.8	0.8
900	40	0.9	0.9	0.9	1.0	1.0	0.9
1 000	40	0.9	0.9	0.8	0.9	0.9	0.9

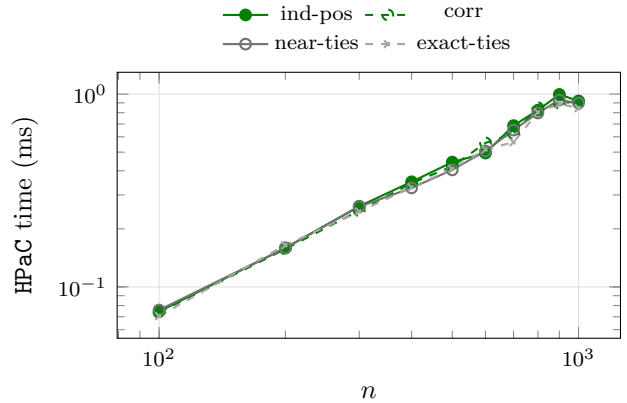


Figure 37: Median CPU time of HPaC in milliseconds, one column per coefficient family and one row per size. Instances: **mixed-forest**, $n \in \{100, \dots, 1000\}$, all four densities, 40 instances per cell. The plot shows four of the six families, the table all six. Observations: at these sizes all times are sub-millisecond and the six families are indistinguishable at the reported resolution – the largest gap at any size is one tenth of a millisecond, i.e. the last digit. The family effect exists but needs $n \geq 10^4$ to be measurable (Fig. 39); this table is here to show that it is *not* measurable below that, so no conclusion in Section 8.1 rests on it.

median RaC CPU time (ms) per family							
n	#inst	anti	corr	exact	ind-pos	ind-sgn	near
100	40	0.3	0.3	0.2	0.3	0.3	0.3
200	40	0.6	0.6	0.6	0.6	0.6	0.6
300	40	0.9	0.9	0.9	0.9	0.9	0.9
400	40	1.3	1.3	1.2	1.3	1.2	1.2
500	40	1.5	1.5	1.4	1.6	1.5	1.5
600	40	1.9	1.8	1.7	1.9	1.9	1.8
700	40	2.2	2.2	2.0	2.2	2.4	2.3
800	40	2.9	2.9	2.7	2.9	2.9	2.8
900	40	3.5	3.2	3.0	3.3	3.3	3.1
1 000	40	3.1	3.0	2.9	3.2	3.1	2.9

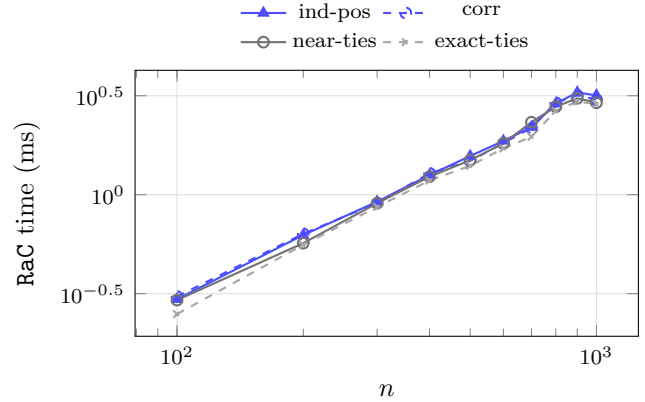


Figure 38: Median CPU time of RaC in milliseconds, one column per coefficient family and one row per size. Instances: **mixed-forest**, $n \in \{100, \dots, 1000\}$, all four densities, 40 instances per cell. The plot shows four of the six families, the table all six. Observations: the same picture as Fig. 37 a factor of three higher, with the families within 0.5ms of each other at $n = 1000$. Every family shows the same dip from $n = 900$ to $n = 1000$ (3.5 to 3.1ms for **anti-correlated**), and so does HPaC, while the number of closure layers keeps growing (470 to 528) and PaC keeps growing with it. We attribute the two elevated sizes to memory-bandwidth contention between the parallel lanes of the sweep (Section 4), which touches the two memory-bound algorithms and not the scan-based one; the effect is $\approx 10\%$, an order of magnitude below every ratio reported in this document, and it is visible only in this per-cell breakdown.

Code, generators, raw and processed data: <https://github.com/fabiofurini/parametric-closure-forests>, release v0.3.0 (instance archives split under GitHub’s asset size limit, with SHA-256 checksums). The pseudoflow baseline is the unmodified upstream implementation at <https://github.com/hochbaumGroup/Bounded-precision-simple-parametric.git>.

		median HPaC CPU time (ms) per family					
n	#inst	anti	corr	exact	ind-pos	ind-sgn	near
10 000	40	13.4	13.2	12.3	12.4	13.0	12.0
20 000	40	29.1	28.7	27.1	29.8	29.6	27.3
30 000	40	49.6	47.5	46.0	46.5	48.5	42.4
40 000	40	66.4	65.4	61.6	66.0	64.7	61.2
50 000	40	91.1	88.6	80.7	93.1	87.9	83.2
60 000	40	100.4	98.9	92.8	99.5	100.1	93.2
70 000	40	116.5	114.9	114.9	124.4	125.5	115.5
80 000	40	144.8	143.9	135.3	149.9	149.2	134.4
90 000	40	174.9	172.8	161.0	174.2	183.6	157.9
100 000	40	229.5	217.3	202.3	224.7	224.2	196.3

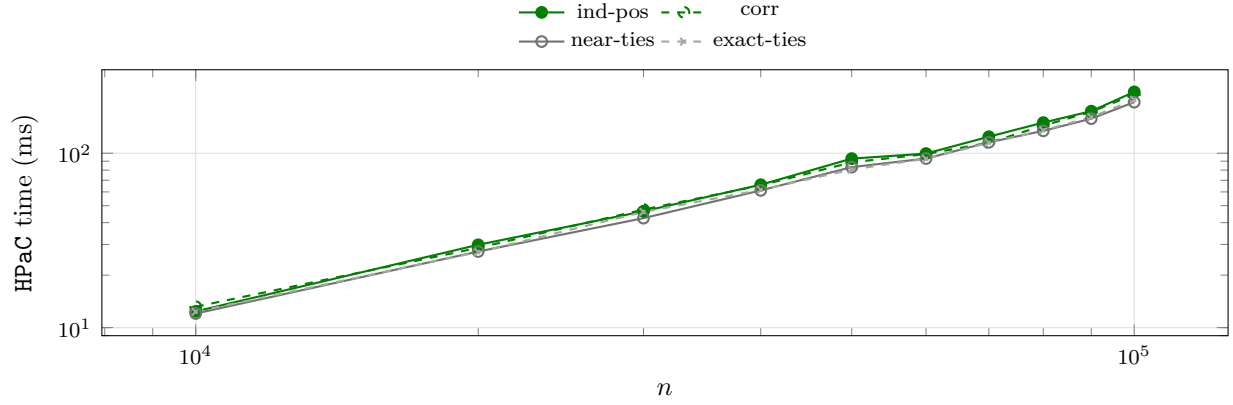


Figure 39: Median CPU time of HPaC in milliseconds, one column per coefficient family and one row per size. Instances: **mixed-forest**, $n \in \{10\,000, \dots, 100\,000\}$, all four densities, 40 instances per cell. The plot shows four of the six families, the table all six. Observations: here the family effect is measurable and consistent – the two tie-stressing families (**near-ties**, **exact-ties**) occupy the two fastest columns at 9 of the 10 sizes, and **anti-correlated** or **independent-signed** the slowest – with a spread between fastest and slowest family of 8–17% that neither grows nor shrinks with n . Compare Fig. 4: **near-ties** produces $27\times$ fewer closure layers than **independent-signed** at these sizes, yet costs only about 10% less time – so the per-layer cost is far from constant, and most of HPaC’s work at $n = 10^5$ is proportional to n (building and rebuilding the heaps) rather than to the number of breakpoints.

		median RaC CPU time (ms) per family					
n	#inst	anti	corr	exact	ind-pos	ind-sgn	near
10 000	40	52.8	53.0	49.4	53.3	53.1	49.8
20 000	40	143.2	142.6	132.1	142.2	142.1	129.1
30 000	40	266.5	267.8	242.2	263.5	265.7	238.7
40 000	40	409.0	402.0	366.8	409.2	412.7	357.4
50 000	40	558.3	535.2	484.4	551.0	548.1	481.0
60 000	40	669.3	649.0	602.8	668.8	663.7	582.7
70 000	40	831.8	824.6	839.3	835.4	832.4	779.2
80 000	40	1 036.9	1 040.8	1 020.4	1 036.0	1 034.7	1 027.5
90 000	40	1 318.6	1 329.2	1 311.8	1 337.4	1 329.0	1 304.3
100 000	40	1 768.8	1 792.4	1 735.5	1 765.6	1 753.1	1 701.3

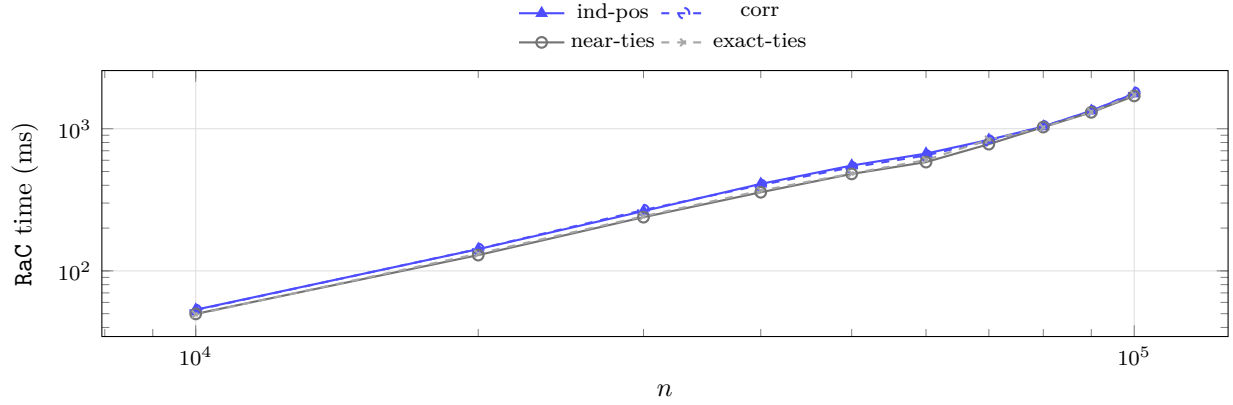


Figure 40: Median CPU time of RaC in milliseconds, one column per coefficient family and one row per size. Instances: **mixed-forest**, $n \in \{10\,000, \dots, 100\,000\}$, all four densities, 40 instances per cell. The plot shows four of the six families, the table all six. Observations: the family spread is 5–11%, comparable to HPaC’s, and the ordering is the same, which is why the paired ratio of Fig. 17 is flat: the two algorithms react to the coefficients in the same way. RaC’s growth over this range is the steepest of any algorithm here – a factor 33 (53 to 1 766 ms) for a $10\times$ increase in n , against a factor 17 for HPaC – which is consistent with its being the only algorithm whose space is $\mathcal{O}(n \log n)$: past a few tens of thousands of vertices its top-tree structures no longer fit in cache and every contraction round pays main-memory latency. We report this as an interpretation, not as a measurement: we did not instrument cache misses.